

B.M.S. COLLEGE OF ENGINEERING BENGALURU
Autonomous Institute, Affiliated to VTU



Lab Record

Object-Oriented Modeling

Submitted in partial fulfillment for the 5th Semester Laboratory

Bachelor of Engineering
in
Computer Science and Engineering

Submitted by:

Akshat Basra

1BM23CS020

Department of Computer Science and Engineering
B.M.S. College of Engineering
Bull Temple Road, Basavanagudi, Bangalore 560 019
August 2025-December 2025

B.M.S. COLLEGE OF ENGINEERING
DEPARTMENT OF COMPUTER SCIENCE AND
ENGINEERING



CERTIFICATE

This is to certify that the Object-Oriented Modelling (22CS5PCOOM) laboratory has been carried out by **Akshat Basra (1BM21CS020)** during the 5th Semester August 2025-December 2025

.

Signature of the Faculty Incharge
Megavalli M
Assistant Professor
Department of Computer Science and Engineering
B.M.S. College of Engineering, Bangalore

Table of Contents

SI NO.	Application
1	Hotel Management System
2	Credit Card Processing System
3	Library Management System
4	Stock Maintenance System
5	Passport Automation System

1. Hotel Management System

1.1 SRS

Problem Statement

A software for Hotel Management System.

1. Introduction

1.1 Purpose

To develop an automated system that streamlines hotel operations, including room booking, customer management, billing, and staff coordination, thereby improving service efficiency, reducing manual work, and enhancing guest satisfaction.

1.2 Scope

The system enables room booking (online/offline), guest check-in/check-out, reservation management, billing & payments, inventory tracking (amenities, housekeeping supplies), staff scheduling, and report generation. It is intended for receptionists, managers, housekeeping, and customers.

1.3 Overview

The software automates core hotel operations, optimizing room allocation, customer service, and internal workflow, while maintaining secure data management and a user-friendly interface.

2. General Description

- Online/Offline room reservation
- Guest check-in/check-out management
- Billing and payment processing
- Room availability and occupancy tracking
- Housekeeping and maintenance monitoring
- Inventory and resource management

- Staff scheduling and assignment
- Report generation (daily/monthly)

3. Functional Requirements

- User registration/login (Admin, Staff, Customer)
- Room booking and cancellation
- Check-in and check-out processing
- Automated billing and invoice generation
- Real-time room status update
- Inventory and housekeeping management
- Staff and shift management
- Email/SMS booking confirmation and alerts
- Report generation (occupancy, revenue, feedback)

4. Interface Requirements

- Web/Mobile responsive interface
- Secure login and access control
- Online payment integration (UPI, Credit/Debit Cards)
- Real-time room availability dashboard
- Email/SMS API connectivity
- Database integration for records and booking history

5. Performance Requirements

- Support minimum 3,000 concurrent users
- Process booking requests within 2 seconds
- Generate billing within 1 second
- System uptime of 99.8%
- Scalable for multiple branches

6. Design Constraints

- Compliance with hotel regulatory & financial standards
- Encrypted storage for personal and payment data
- Integration with third-party payment gateways
- Cross-platform compatibility
- Must support both cloud and local deployment

7. Non-Functional Attributes

- High-security standards (SSL/TLS, encryption)
- User-friendly and intuitive interface
- Scalable for future expansion
- Daily backup & recovery options
- Maintainable with modular architecture
- Multilingual support (optional)

8. Preliminary Schedule & Budget

Phase	Duration (weeks)	Cost (₹)
Requirements Gathering	3	2,50,000
System Design	4	3,50,000
Development	10	15,00,000
Testing	3	3,00,000
Deployment	2	1,50,000
Training & Documentation	1	1,00,000
Contingency (10–15%)	—	2,50,000
Total	23 weeks	29,00,000

```

classDiagram
    class Payment {
        +bno: int
        +computebill()
    }
    class Hotel {
        +id: int
        +hname: string
        +hno: int
        +checkRA(): boolean
        +broker(): int
        +generatebill(): float
    }
    class Branch {
        +bname: string
        +baddr: string
    }
    class Customer {
        +cust_no: string
        +mobile: int
        +address: string
        +checkin(): int
        +checkout(): int
    }
    class Room {
        +rno: int
        +rtype: string
    }
    class Worker {
        +salary: float
        +compute_salary()
    }
    class Employee {
        +id: int
        +empna: string
        +empdob: date
        +hiremp()
        +firemp()
    }
    class Stock {
        +sname: string
        +id: int
        +checkstate()
    }
    class Food {
        +itemid: int
        +ano: int
        +cancel(): int
        +book(): int
    }
    class chef {
        +id
        +name
        +address
        +cookfood()
    }
    class manager {
    }
    class Receptionist {
    }
    class Kitchen {
    }
    class DiningRoom {
    }
    class Balcony {
    }
    class RR {
    }

    Payment "*" -- "*" Customer : pays
    Customer "1" -- "*" Room : books
    Customer "*" -- "1" Employee : takes feedback
    Hotel "*" -- "*" Employee : employs
    Employee "1" -- "*" Stock : orders
    Employee <|-- chef
    Employee <|-- manager
    Employee <|-- Receptionist
    Food "*" -- "*" chef : makes
    Branch "1" -- "1" Hotel : hotel_id
    Room <|-- RR
    Room <|-- Balcony
    Room <|-- DiningRoom
    
```

Room Type

- +Single-AC
- +Single-nonAC
- +double-AC
- +double-nonAC

Payment Mode

- +cash
- +credit card
- +debit card
- +phonepe
- +gpay
- +stripe

The class diagram represents the structure of the Hotel Management System, showing the key entities involved in hotel operations and their relationships.

- Hotel: Stores hotel details and manages bill generation and room availability.
- Branch: Represents different hotel branches with name and address.
- Customer: Holds customer information such as contact and address; performs check-in/check-out.
- Room: Represents rooms with room number and type; specialized into R R, Balcony, and Dining Room.
- Payment: Manages billing information and computes bills; associated with customers.

- Employee: Base class for staff with details like name, ID, and date of birth. Includes operations for hiring and removal.
 - Specialized into Manager, Receptionist, Chef, and Worker.
- Food: Represents food items that can be ordered; linked with Chef.
- Kitchen: Supports food preparation.
- Stock: Manages stock items and their state.

Enumerations

- Room Type: Single-AC, Single-nonAC, Double-AC, Double-nonAC.
- Payment Mode: Cash, credit card, debit card, phonepe, gpay, stripe.

Relationships

- Hotel–Branch: A hotel has multiple branches.
- Hotel–Employee: A hotel employs many employees.
- Customer–Room: Customers book rooms.
- Customer–Payment: Payments are made by customers for services.
- Employee–Food: Chefs prepare food; Receptionists take orders.
- Employee–Stock: Employees check and manage stock.
- Customer–Employee: Customers provide feedback to employees.

1.3 State Diagram:

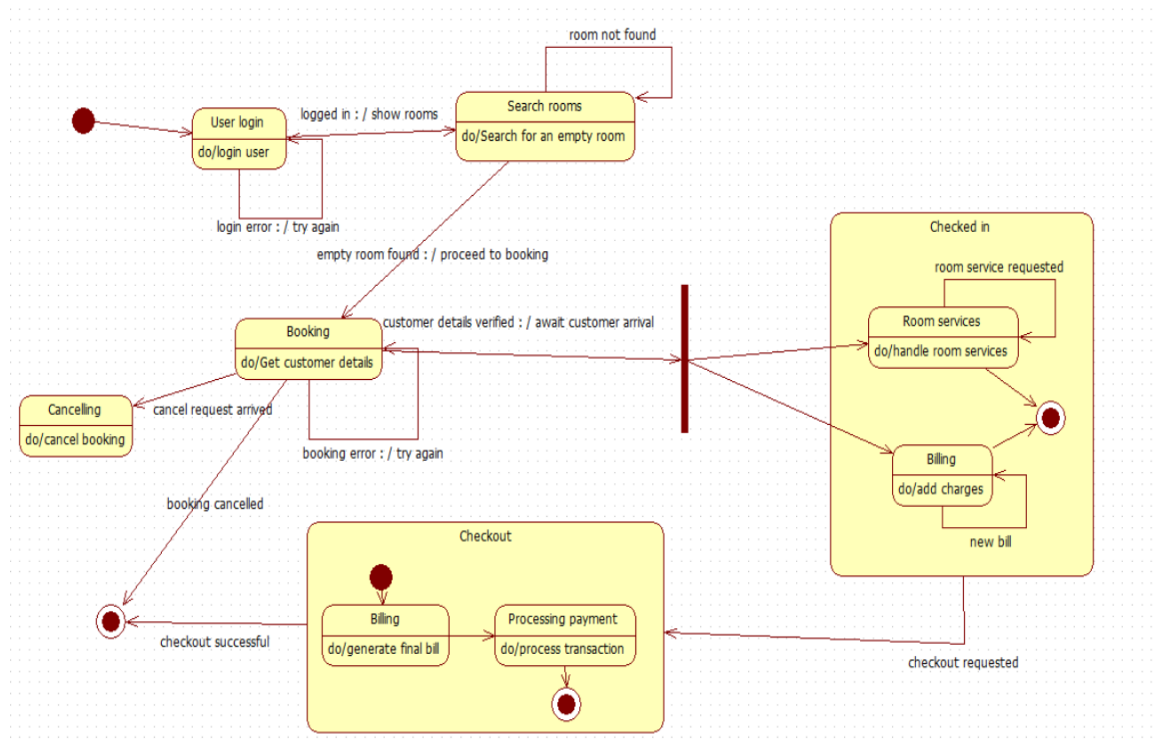


Fig 1.2 State Diagram

Description

The state diagram illustrates the sequence of activities in the Hotel Booking and Check-in process, showing how the system manages user login, room search, booking, room services, billing, and checkout.

States

- **User Login:** The user enters credentials and is either logged in or prompted to retry on error.
- **Search Rooms:** The system searches for empty rooms; if no room is found, the user is asked to retry.
- **Booking:** Customer details are collected and validated to confirm the room booking.
- **Cancelling:** Triggered when the customer cancels a booking request.
- **Checked In (Composite State):** Represents the guest's stay, including room services and billing updates.

- Includes substates: **Room Services** and **Billing** for adding charges.
- **Checkout (Composite State):** Handles final bill generation and processing of payment.
- Includes substates: **Billing** and **Processing Payment**.
- **End State:** The process completes after checkout is successful.

Flow

1. User logs in; incorrect details lead to a retry.
2. The system searches for an empty room; if found, proceeds to booking.
3. Booking verifies customer details; failure leads to reattempt, and cancellation ends the booking.
4. After successful booking, the system awaits customer arrival and transitions to the checked-in state.
5. During the stay, customers may request room services, and additional charges are added to the bill.
6. Customer requests checkout, triggering bill generation and payment processing.
7. After successful payment, the checkout is completed.

1.4 Use Case Diagram

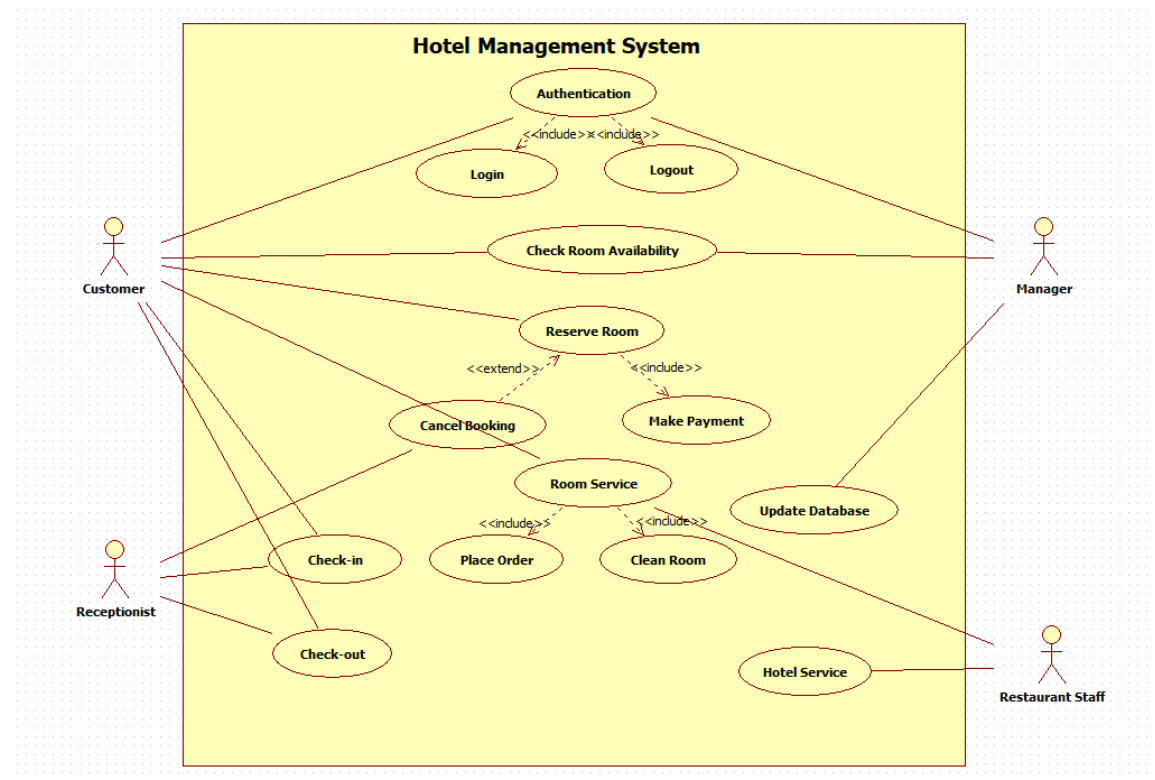


Fig 1.3 Use Case Diagram

Description

The use case diagram represents the interactions between the main actors of the Hotel Management System and the services provided by the system. It shows how customers, receptionists, managers, and restaurant staff collaborate to handle room reservations, check-in/check-out, payments, room services, and hotel services.

Actors

- **Customer** : Logs into the system, checks room availability, reserves or cancels rooms, requests room services, places orders, and makes payments.
- **Receptionist** : Manages guest check-in and check-out and coordinates front-desk operations.
- **Manager** : Monitors room availability and updates the hotel database based on reservations and payments.
- **Restaurant Staff** : Provides hotel/restaurant services and fulfills customer orders.

Use Cases

Customer

- **Authentication** : Overall process that includes Login and Logout.
- **Login** : Allows the customer to access the system.
- **Logout** : Ends the customer session.
- **Check Room Availability** : Views available rooms before booking.
- **Reserve Room** : Books a room; includes Make Payment and is extended by Cancel Booking.
- **Cancel Booking** : Extends Reserve Room when the customer decides to cancel a reservation.
- **Make Payment** : Handles payment for room reservation and other services; updates records through Update Database.
- **Room Service** : Handles service requests during the stay; includes Place Order and Clean Room.
- **Place Order** : Included use case for ordering food or other services through Room Service.

Receptionist

- **Check-in** : Registers the guest's arrival and assigns a room.
- **Check-out** : Processes guest departure and final settlement.

Manager

- **Update Database** : Keeps room, booking, and payment information up to date; typically triggered after Make Payment and other operations.
- **Check Room Availability** : May be monitored or managed by the Manager for operational decisions.

Restaurant Staff

- **Hotel Service** : Delivers food and other hospitality services based on customer orders and room service requests.

1.5 Sequence Model

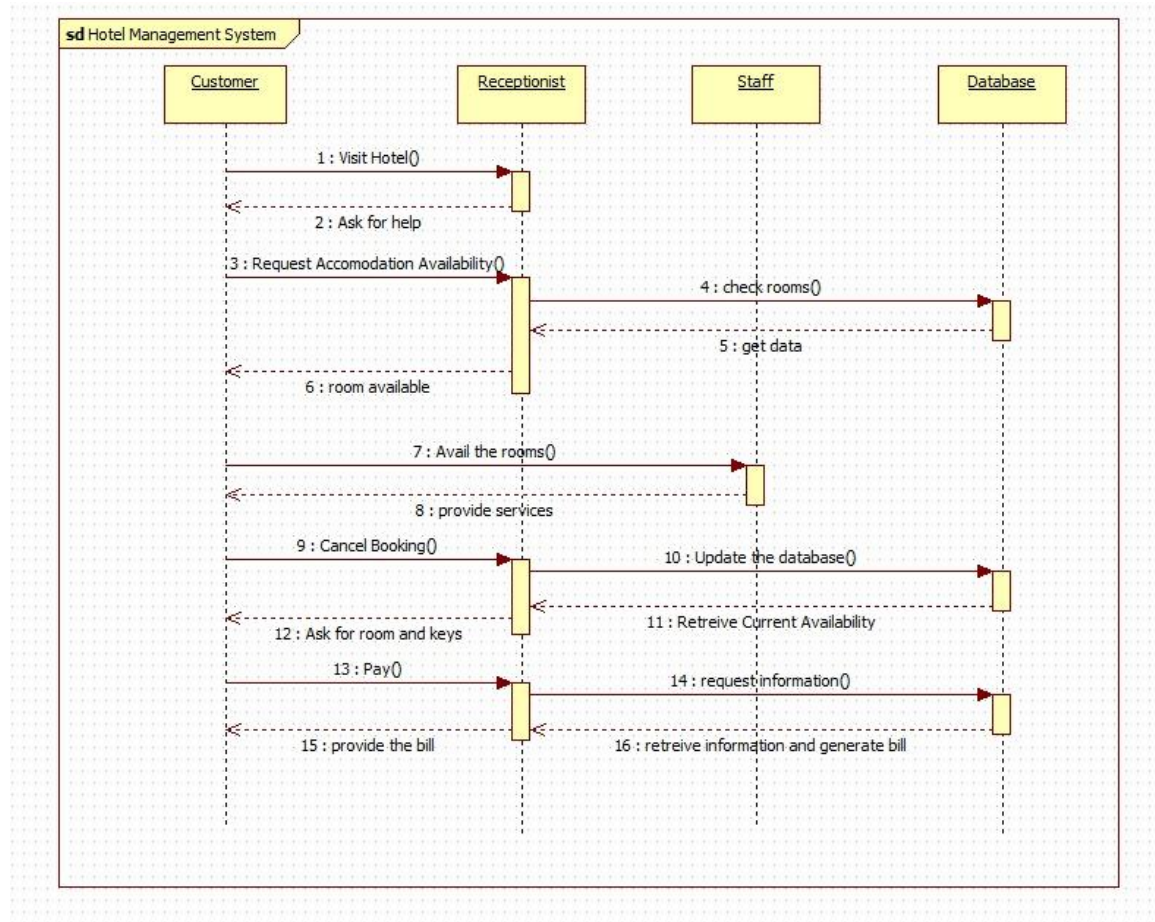


Fig 1.4 Sequence Model

Process steps:

1. Initial Inquiry:
 - Customer asks the Receptionist for help and requests room availability.
2. Availability Check & Booking:
 - The Receptionist checks room status by sending check rooms() to the Database.
 - The Receptionist informs the Customer of availability.
 - The Customer confirms the booking, and the Staff begins to provide services.
3. Cancellation:
 - The Customer sends Cancel Booking().
 - The Receptionist ensures the Database is updated.
4. Room Access:

- The Receptionist retrieves Current Availability from the Database before the Customer asks for keys.

5. Payment & Billing:

- The Customer sends Pay() to the Receptionist.
- The Receptionist retrieves information from the Database, generates the bill, and then sends the bill to the Customer.

1.6 Activity Model

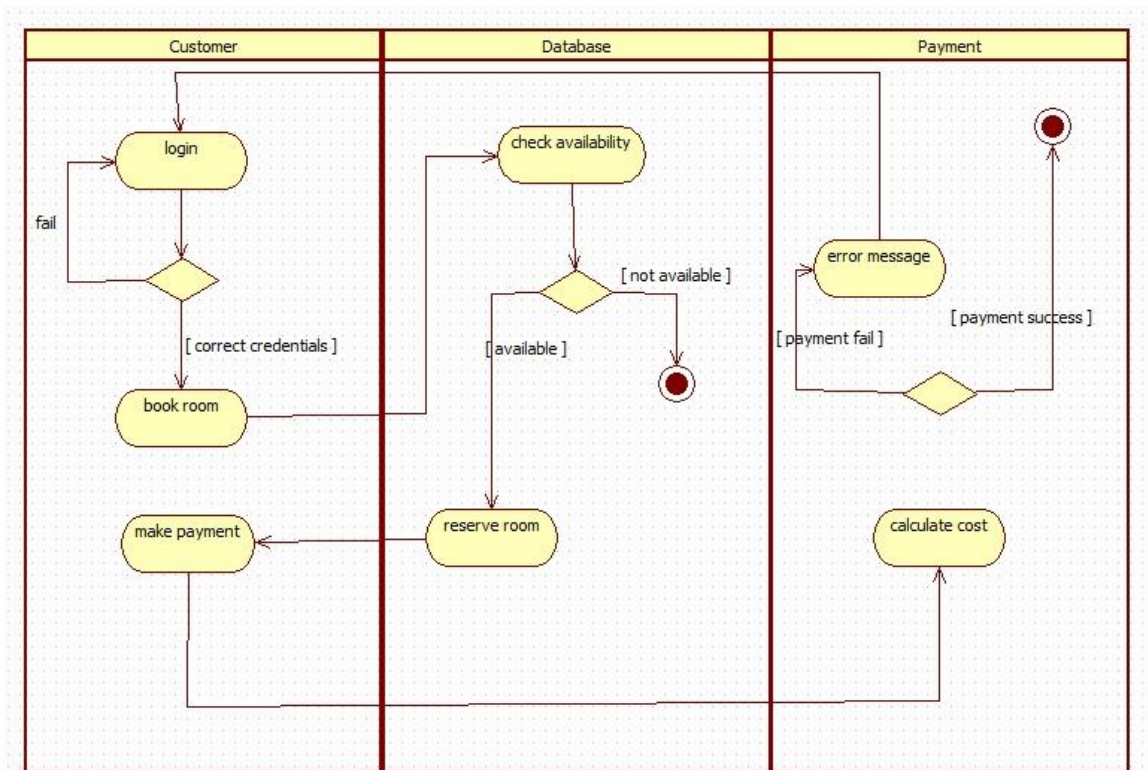


Fig. 1.5 Activity Diagram

Customer Swimlane

- Login activity is initiated by the customer.
 - A decision node evaluates credentials:
 - Fail - flow terminates.
 - Correct credentials - proceed to booking.
- Book Room request is sent to the database.
- After room reservation, the customer performs Make Payment, forwarding the process to the payment system.

Database Swimlane

- Upon booking request, the system performs Check Availability.

- Decision node evaluates room availability:
 - Not available - end node reached.
 - Available - execute Reserve Room and pass control to payment.

Payment Swimlane

- Calculate Cost is performed upon receiving the reservation details.
- Decision node evaluates payment result:
 - Payment fail - display Error Message and terminate.
 - Payment success - reach the final node indicating successful completion.

2. Credit Card Automation System

2.1. SRS

2.2 Class Model

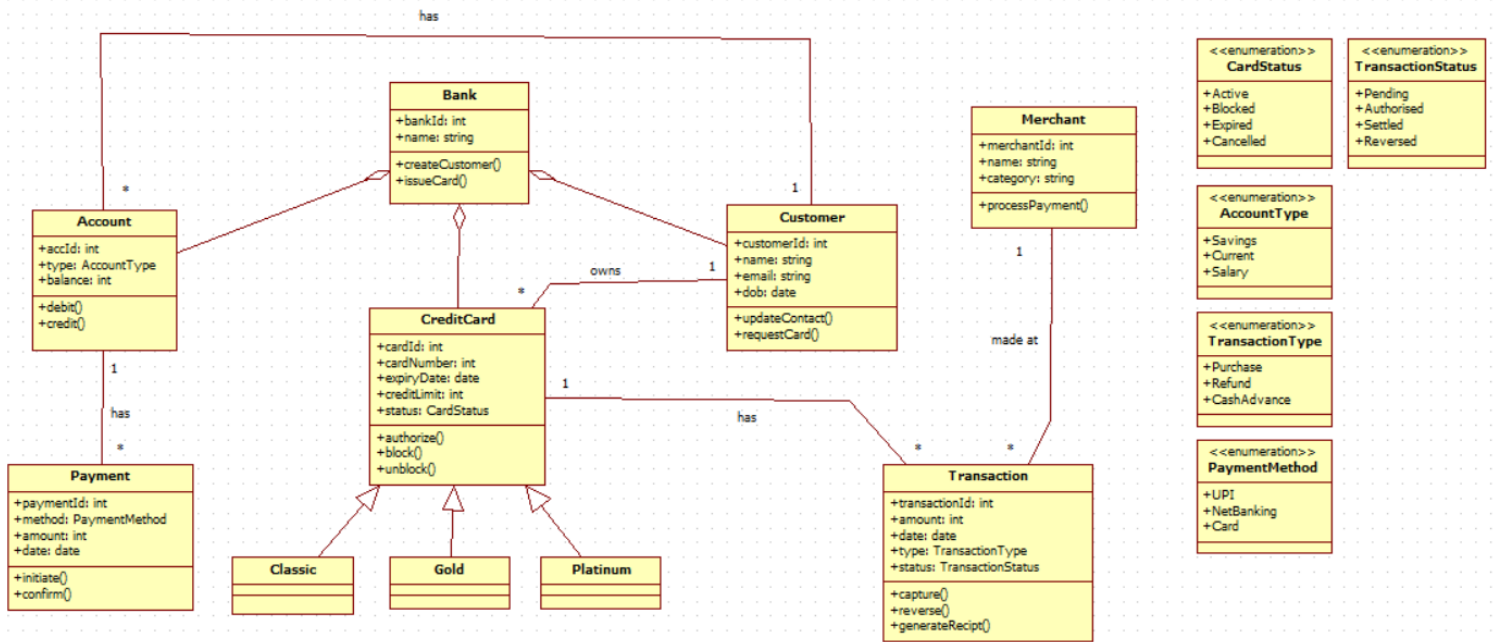


Fig 2.2 Class Model

A bank is the central entity, identified by bankId and name. It manages customers, represented by the customerId and has an operation to issueCard(). It has a one-to-many association with Account (a Bank *has* multiple Accounts) and a one-to-many association with CreditCard (a Bank *issues* multiple CreditCards).

Customer is identified by customerId, name, email, and dob. It has operations to updateContact() and requestCard(). A Customer has a one-to-one relationship with a Bank (a Customer *belongs to* one Bank). A Customer *owns* one or more CreditCard instances.

CreditCard is a key entity, defined by cardId, cardNumber, expiryDate, creditLimit, and status (from the CardStatus enumeration). It includes operations to block() and unblock(). The specific types of credit cards—Classic, Gold, and Platinum—are modeled as subclasses of CreditCard using inheritance.

An Account is linked to a Payment via a one-to-one association (an Account *has* a Payment). An Account has accId, balance, and type (from the AccountType enumeration), along with operations for debit() and credit().

Transaction represents the financial movement associated with a Merchant (a Transaction is *made at* one Merchant) and a CreditCard (a CreditCard *has* multiple Transactions). It includes transactionId, amount, date, type (from TransactionType), and status (from TransactionStatus). Operations include capture(), reversal(), and generateReceipt().

Merchant is associated with a Transaction (a Merchant *is involved in* multiple Transactions). It includes merchantId, name, and category, and has the operation processPayment().

Payment is a transactional entity with paymentId, method (from PaymentMethod), amount, and date. It provides operations to initiate() and confirm().

2.3 State Model

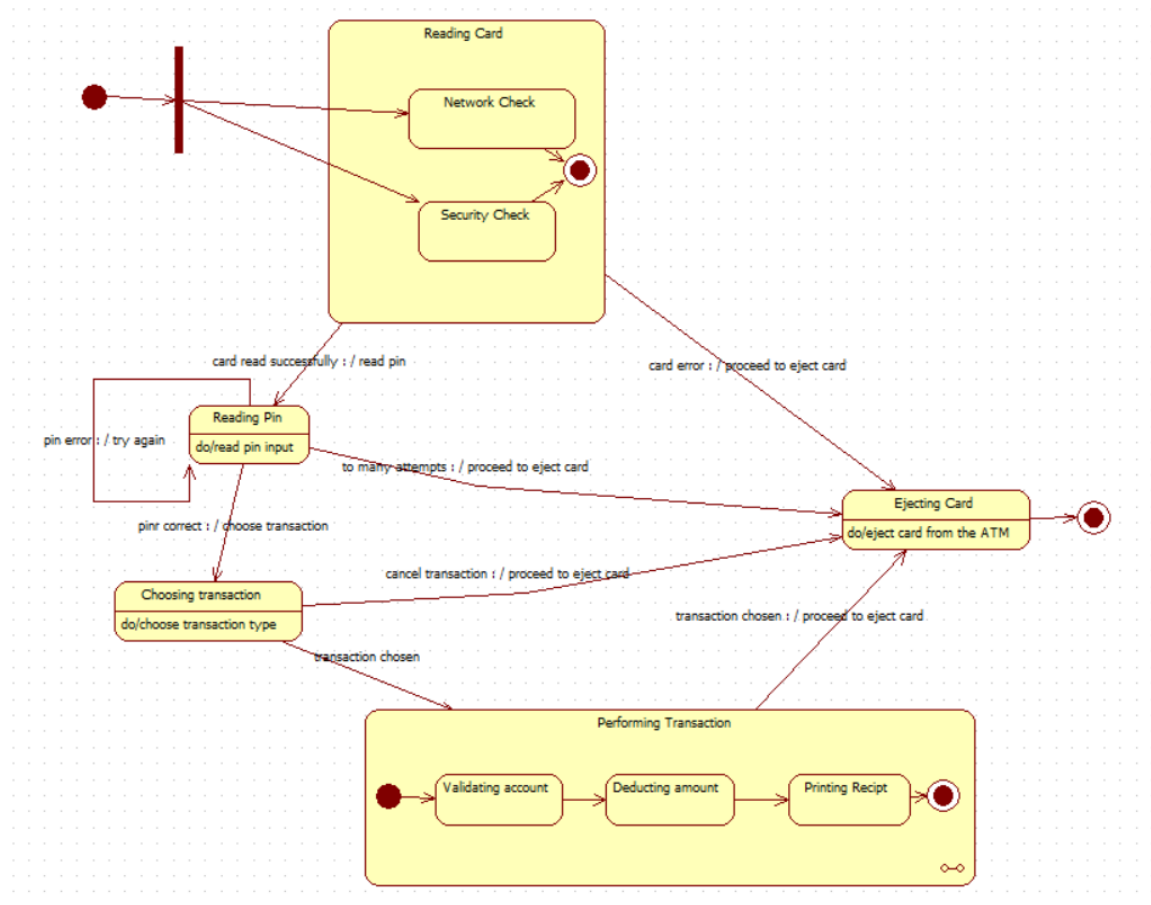


Fig 2.3 State Model

- Reading Card: This activity includes parallel checks (Network Check and Security Check).
 - a. If successful, proceeds to Reading Pin.
 - b. If a card error occurs, it proceeds immediately to Ejecting Card.
- Reading Pin: The user inputs their PIN.
 - a. Pin error loops back to try again.
 - b. If to many attempts occur, it proceeds to Ejecting Card.
 - c. If the pin is correct, it proceeds to Choosing Transaction.
- Choosing Transaction: The user selects a service.
 - a. If the user chooses to cancel transaction, it proceeds to Ejecting Card.
 - b. Once a transaction chosen, it proceeds to Performing Transaction (or Ejecting Card per the specific transition label shown).
- Performing Transaction: This group details the execution steps: Validating account, Deducting amount, and Printing Receipt.
 - a. Upon completion, it transitions to Ejecting Card.

- Ejecting Card: The final action where the ATM physically returns the card (do/eject card from the ATM) before the process terminates.

2.4 Use Case Model

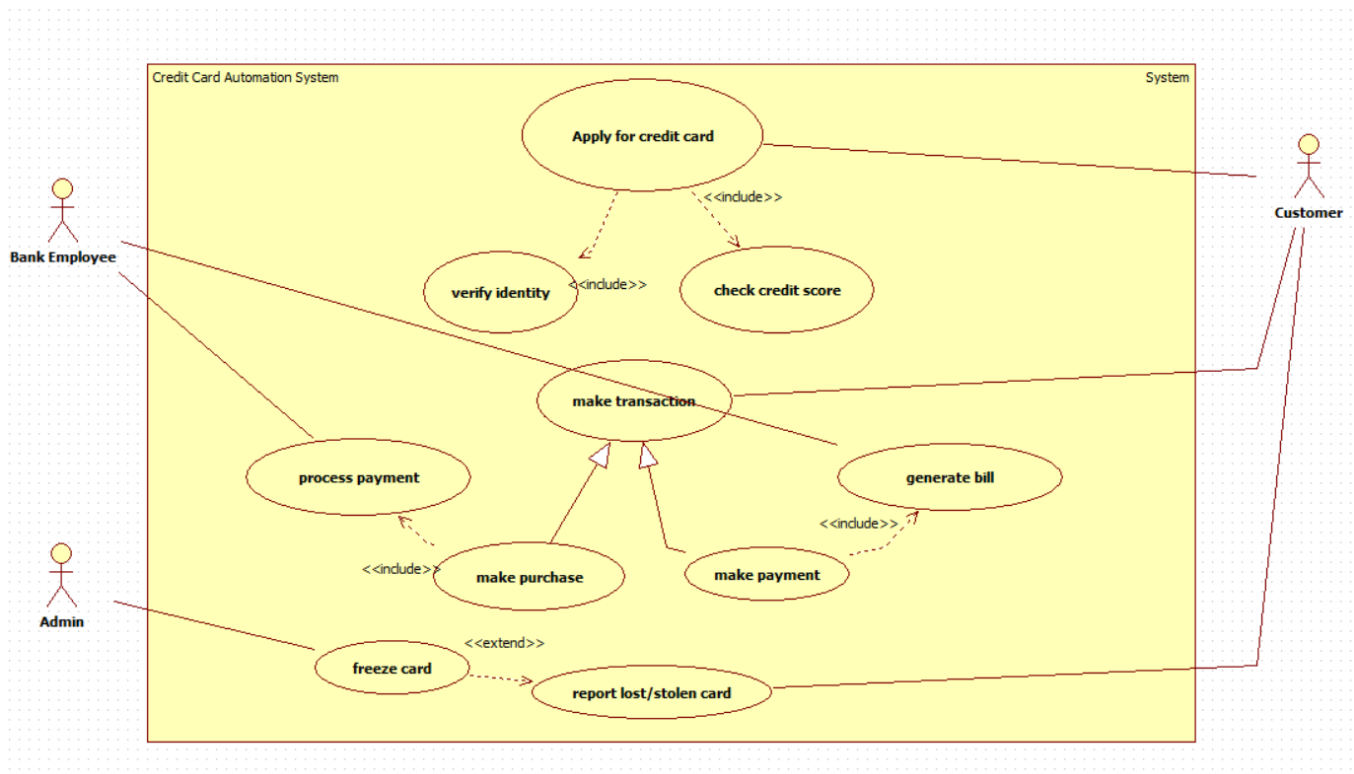


Fig 2.4 Use Case Model

Actors:

- **Customer:** The primary user who interacts directly with the core credit card services.
- **Bank Employee:** Handles customer verification and payment processing tasks.
- **Admin:** Responsible for security and control functions, such as freezing a card.
- **System:** Represents automated processes within the application (e.g., generating a bill or checking a credit score).

Use cases:

1. Card Application and Setup

- Apply for credit card – Initiated by the customer.
 - Includes verify identity (performed by bank employee)
 - Includes check credit score (performed by the system)

2. Transaction Management

- Make transaction – Abstract use case; serves as a generalization for specific transaction types.
- Make purchase – Initiated by the customer; includes process payment.
- Make payment – Customer pays outstanding bill; includes generate bill.

3. Financial Processing

- Process payment – Included in *make purchase*; performed by bank employee.
- Generate bill – Included in *make payment*; performed by the system.

4. Security and Maintenance

- Freeze card – Initiated by admin to temporarily suspend card usage.
 - Can be extended by report lost/stolen card.
- Report lost/stolen card – Initiated by the customer; may trigger *freeze card*.

2.5 Sequence Model

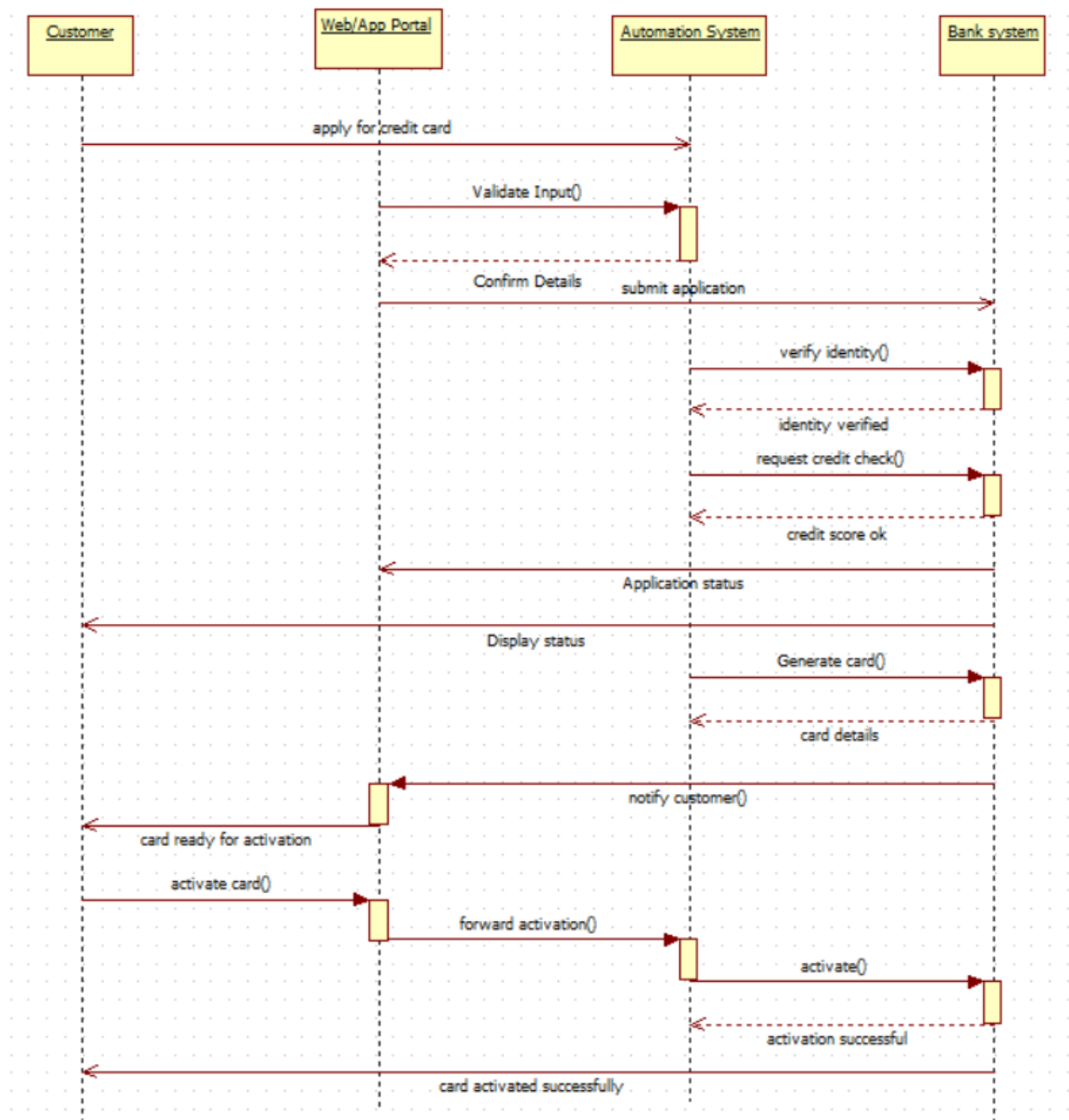


Fig 2.5 Sequence Model

Process Steps:

- Application: The Customer submits the application via the Web/App Portal. The Automation System validates input and forwards the submission to the Bank System.
- Verification: The Bank System performs identity verification and a credit check.
- Approval: If approved, the Bank System confirms status to the Automation System, which updates the Customer via the Web/App Portal.

- Card Generation: The Automation System triggers the Bank System to generate the card, and its details are returned. The Customer is then notified that the card is ready.
- Activation: The Customer requests to activate card(). The request is passed through the Web/App Portal and Automation System to the Bank System, which executes the activation and sends the final success confirmation back to the Customer.

2.6 Activity Diagram

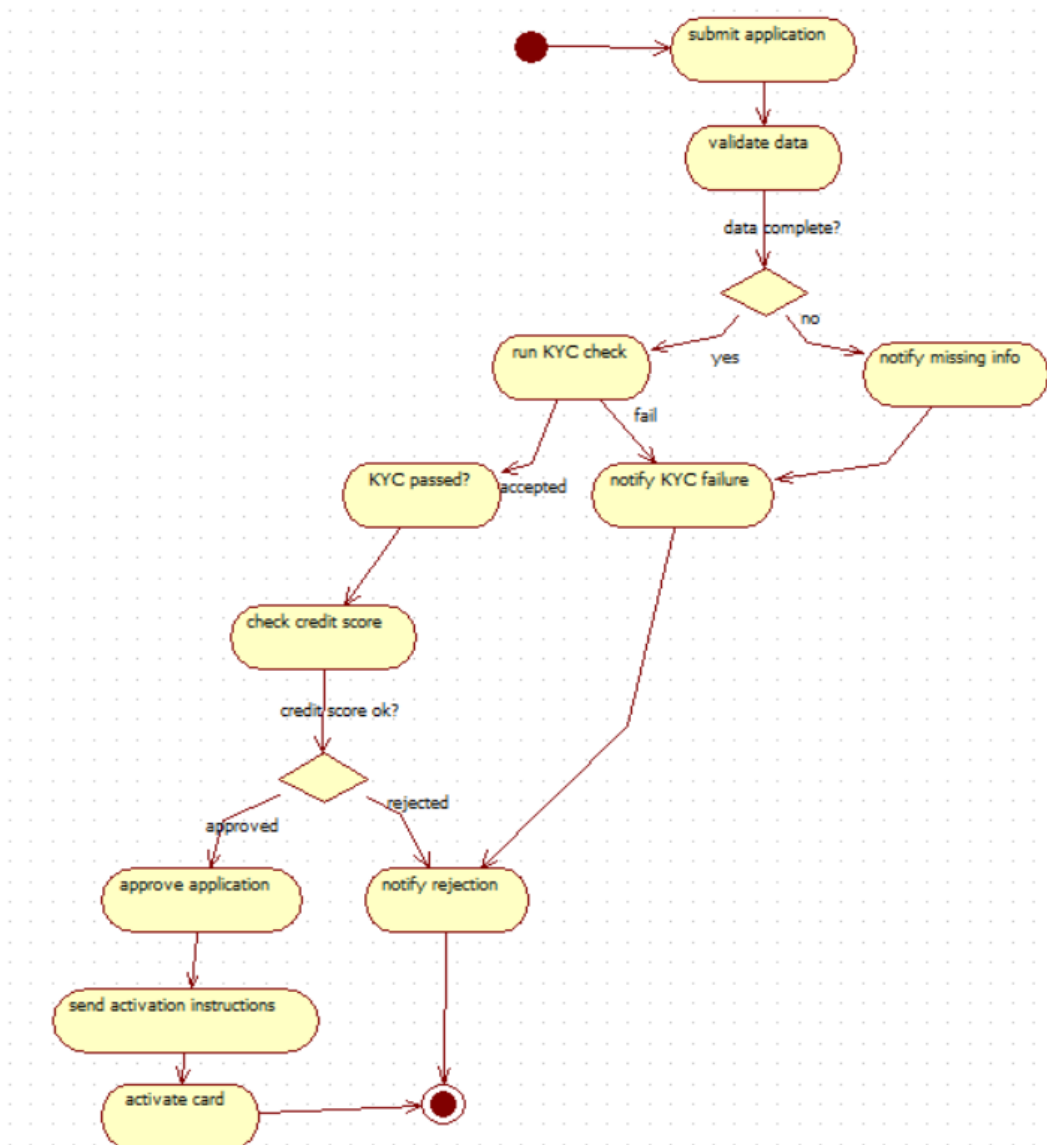


Fig 2.6 Activity Diagram

Flow Summary

1. Submission & Validation: The process starts with the application submission, followed by data validation. If data is incomplete, the applicant is notified and the application fails.
2. Verification: If data is complete, a KYC check is run. Failure here leads to rejection.
3. Credit Check: If KYC is accepted, the system checks the credit score.
4. Decision & Outcome:
 - If the score is rejected, the applicant receives a rejection notification.

- If approved, the application is approved, activation instructions are sent, and the card is activated.

All failure paths and the final activation step lead to the process termination.

3. Library Management System

3.1 SRS

1. Introduction

1.1. Purpose of the document

This document defines the requirements of the Library Management system, which aims to automate various common library tasks.

1.2. Scope

The library management system is a desktop based application to be accessed by librarians, members and admin. Its key functionalities are book keeping record, tracking of due dates, fine calculation etc.

1.3. Overview

The system is intended to automate repetitive tasks like check-in and check-out of books by members, and mathematical calculations like due date and fine calculations. The system is to be implemented in local computers only, and is operated on location. So, the hardware is limited to a single network.

2. General Description

2.1 Objective

The system will act as a centralised system replacing manual registers and records. It will store details of books, members, and transactions in a database, accessible through a simple interface.

2.2 Users

- Members: Customers that borrow and return books.
- Librarians
- Admins.

2.3 Benefits

The system reduces the human hours spent on repetitive tasks of scanning books for issuing and returning of books. Furthermore, it improves efficiency and decreases error in arithmetic calculations.

3. Functional Requirements

3.1 User Management

- The system shall allow librarians and admins to register new members with new IDs.
- The system shall allow login access for members to view issued books and due dates.

- The system shall allow admins to manage librarian accounts.

3.2 Catalog Management

- The system shall allow librarians/admins to add new books with details (title, author, ISBN, category).
- The system shall allow updates to book availability.
- The system shall allow search by title, author, ISBN for category.

3.3 Transaction Management

- The system shall allow librarians to issue books to members with a due date.
- The system shall allow members to return books and automatically update availability.
- The system shall calculate late fees for overdue books.

3.4. Reports and Analytics

- The system shall generate reports on book inventory.
- The system shall generate reports on member activity.
- The system shall allow admins to generate monthly /annual usage and financial reports.

4. Interface Requirements

- An interface is required for members to return books and view/pay late fines.
- An interface is required for librarians to issue books, View member activity and set due dates.
- An interface is required for admins to generate reports.

5. Performance Requirements

- The system should be able to handle 500 users without decline in performance.
- The system must process issuing/returning within 2 seconds.
- The system should be able to generate reports from 10000 records in 20-30 seconds.

6. Design Constraints

The system must comply with institutional policies on data protection. It must be compatible with standard hardware like barcode scanners.

7. Non-Functional Requirements

- Security: Password - protected data access and encrypted member
- Usability: Simple, intuitive interface for both staff and members.

- Performance: Ability to handle at least 500 concurrent users.
- Reliability: 99% uptime and quick recovery.

8. Preliminary Schedule

The system is expected to be completed in about 12 weeks, The phases include requirements gathering, system design, development, testing and deployment.

3.2 Class Model

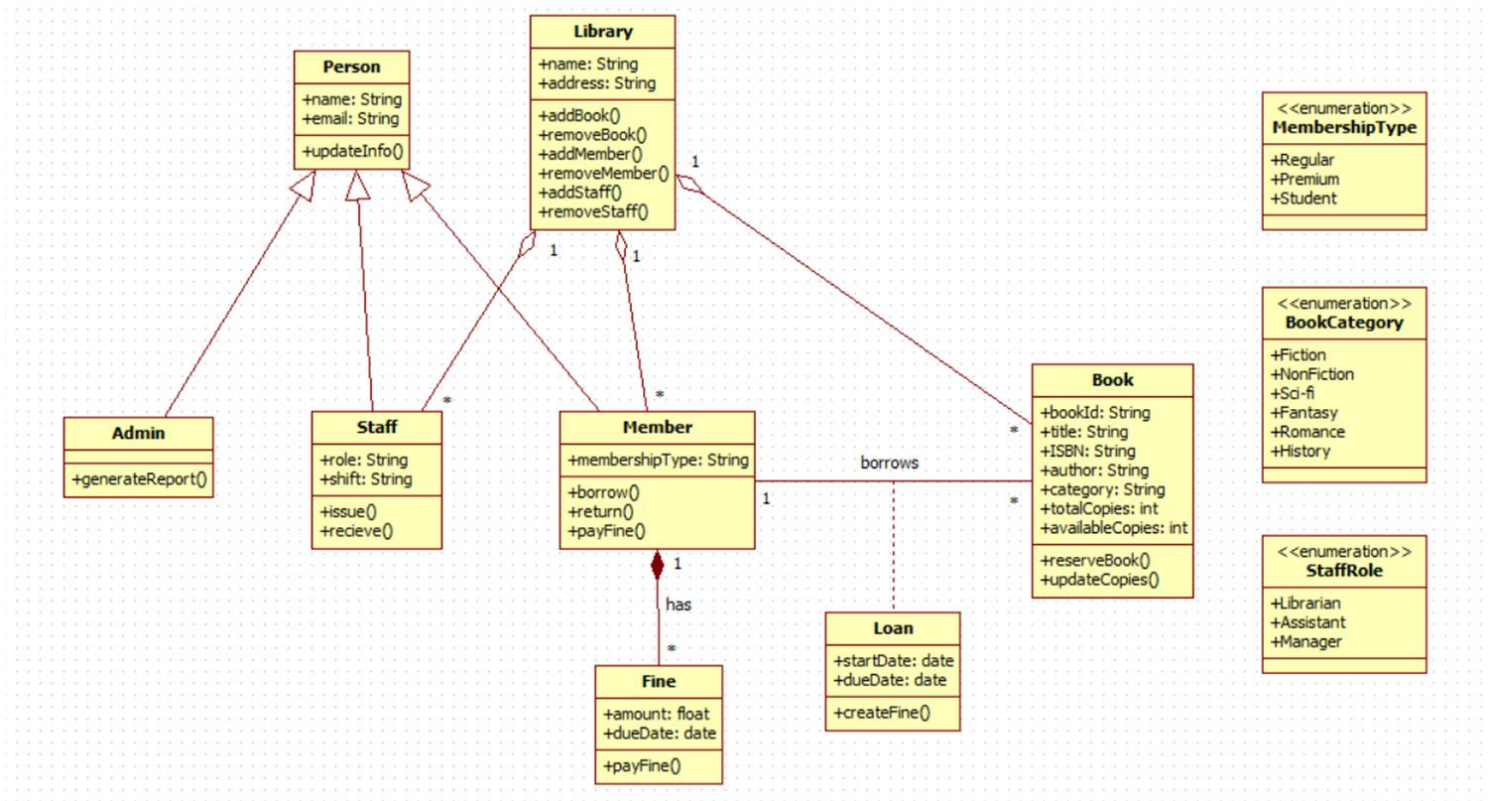


Fig. 3.1 Class Model

- The library serves as the central management entity responsible for maintaining collections of Books, Members, and Staff. It is associated with these classes through aggregation relationships, indicating that while the Library manages these objects, they may exist independently of it. The Library provides operations for adding and removing books, members, and staff.
- Person is defined as an abstract generalized superclass that provides common attributes (name, email) and a shared operation (updateInfo()). Three specialized subclasses extend Person through generalization:
 - Admin, which provides administrative operations such as generating system reports.
 - Staff, which includes role-specific attributes (role, shift) and operations for issuing and receiving books.
 - Member, which encapsulates user-specific information including membership type (defined through the MembershipType enumeration) and supports operations for borrowing books, returning books, and paying fines.
- Book represents a library item with identifying and descriptive attributes such as bookId, title, ISBN, author, category (from the BookCategory enumeration), totalCopies, and availableCopies. It provides operations for reserving books and

updating copy counts. Members interact with Books through a borrows association, with Loan acting as the transactional intermediary.

- Loan represents a borrowing transaction and is linked to Member via a composition relationship, implying that a Loan instance is created and destroyed along with the Member that owns it. Loan records include startDate and dueDate, and provide an operation for initializing fine creation or other loan-related processes.
- Fine is also linked to Member via composition, indicating that fines cannot exist independently of the member to whom they are assigned. A Fine includes attributes such as amount and dueDate, and provides an operation to process or record fine payment.
- Additionally, the StaffRole enumeration provides a controlled vocabulary for roles assigned to staff (Librarian, Assistant, Manager).
- Overall, the structure emphasizes modularity, clarity of responsibility, and strong encapsulation of borrowing and penalty workflows through composition and well-defined associations.

3.3 State Model

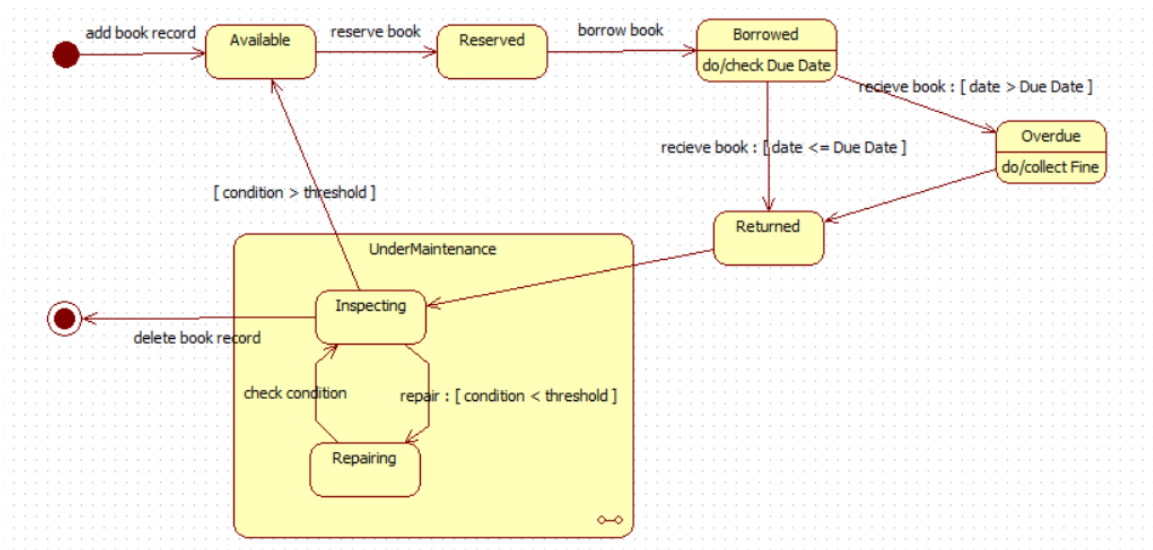


Fig. 3.2 State Model

- A book enters the Available state upon the *add book record* event. From this state, it may be reserved or borrowed.
- When a reservation occurs, the book transitions to Reserved, and subsequently to Borrowed when the user completes the borrowing process.
- In the Borrowed state, the system performs a due-date check. A *receive book* event returns the book to:
 - Returned, if $\text{date} \leq \text{due date}$, or
 - Overdue, if $\text{date} > \text{due date}$, where the system executes fine collection before moving to Returned.
- From Returned or Available, the book may enter the UnderMaintenance composite state if $[\text{condition} > \text{threshold}]$.
- UnderMaintenance contains two substates:
 - Inspecting, which evaluates book condition, and
 - Repairing, entered when repairs are required ($[\text{condition} < \text{threshold}]$). The substates loop until the book passes inspection and exits back to Available.

- A *delete book record* event from any state leads to the Final state, removing the book from the system.

3.4 Use Case Model

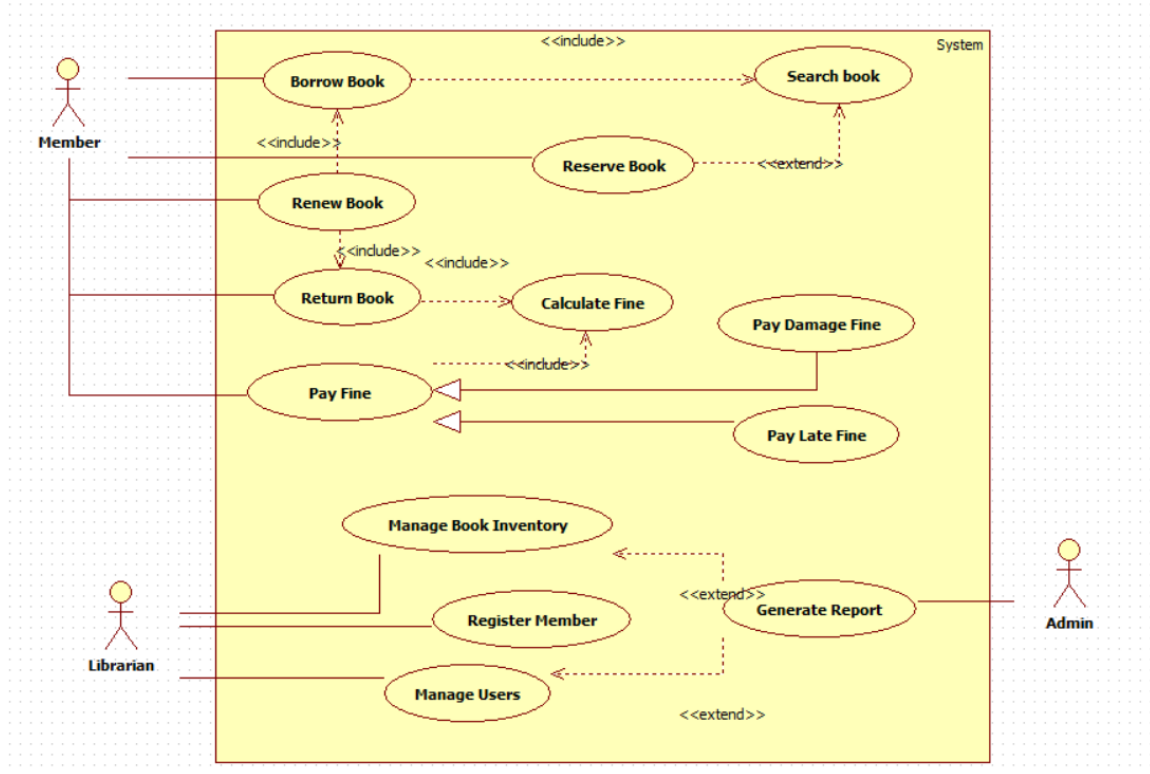


Fig. 3.2 Use Case Diagram

Member Interactions

A **Member** performs core circulation activities such as **Borrow Book**, **Renew Book**, **Return Book**, and **Reserve Book**.

- **Borrow Book** and **Renew Book** both *include* **Search Book**, signifying that searching is a mandatory sub-function.
- **Return Book** *includes* **Calculate Fine**, enabling automatic fine assessment during returns.
- **Pay Fine** is associated with two specialized fine-handling use cases—**Pay Damage Fine** and **Pay Late Fine**—both of which *extend* **Pay Fine**, as these are conditional scenarios.

Librarian Interactions

A **Librarian** oversees administrative tasks, including **Register Member**, **Manage Book Inventory**, and **Manage Users**.

- **Manage Book Inventory** optionally *extends* into **Generate Report**, indicating that reporting is triggered as an optional extension of inventory management activities.

Admin Interactions

The **Admin** actor directly accesses **Generate Report**, which provides system-wide analytical and operational insights.

Cross-Use-Case Dependencies

- **Reserve Book** is an optional extension of **Search Book**, used when a member chooses to reserve a located item.
- **Pay Late Fine** includes **Calculate Fine**, ensuring late fines are computed before payment processing.

3.5 Sequence Diagram

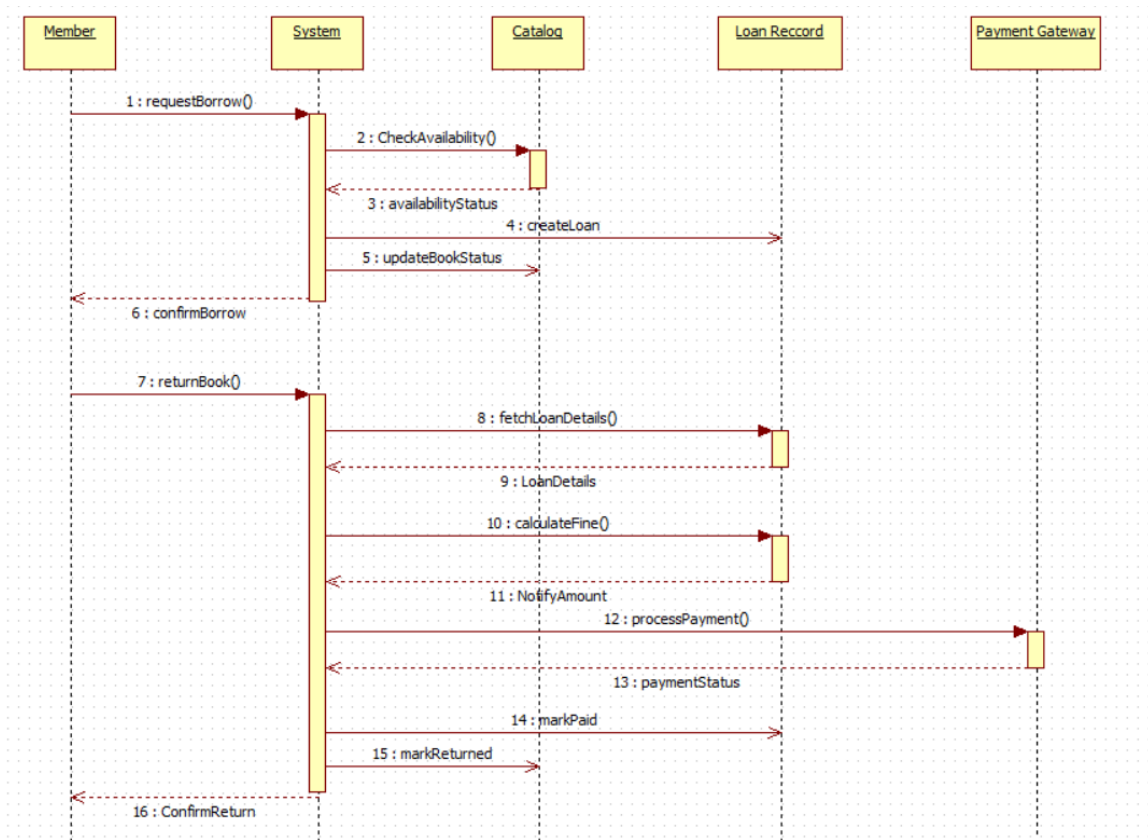


Fig. 3.4 Sequence Model

Borrowing Process

1. The Member initiates borrowing via requestBorrow().
2. The System queries the Catalog through CheckAvailability().
3. The Catalog responds with availabilityStatus.
4. When available, the System instructs the Catalog to createLoan().
5. The System updates the book's status via updateBookStatus().
6. The Member receives confirmBorrow() as confirmation of successful borrowing.

Returning Process

7. The Member triggers the return through returnBook().
8. The System retrieves borrowing details using fetchLoanDetails() from Loan Record.
9. The Loan Record returns LoanDetails.
10. The System calculates applicable fines via calculateFine().
11. The System notifies the Member of due fines through NotifyAmount.

- 12.** If payment is required, the System invokes processPayment() on the Payment Gateway.
- 13.** The Payment Gateway returns the paymentStatus.
- 14.** Upon successful payment, the System marks the loan as paid through markPaid.
- 15.** The System records the item as returned via markReturned().
- 16.** The Member receives ConfirmReturn() to complete the process.

3.6 Activity Model

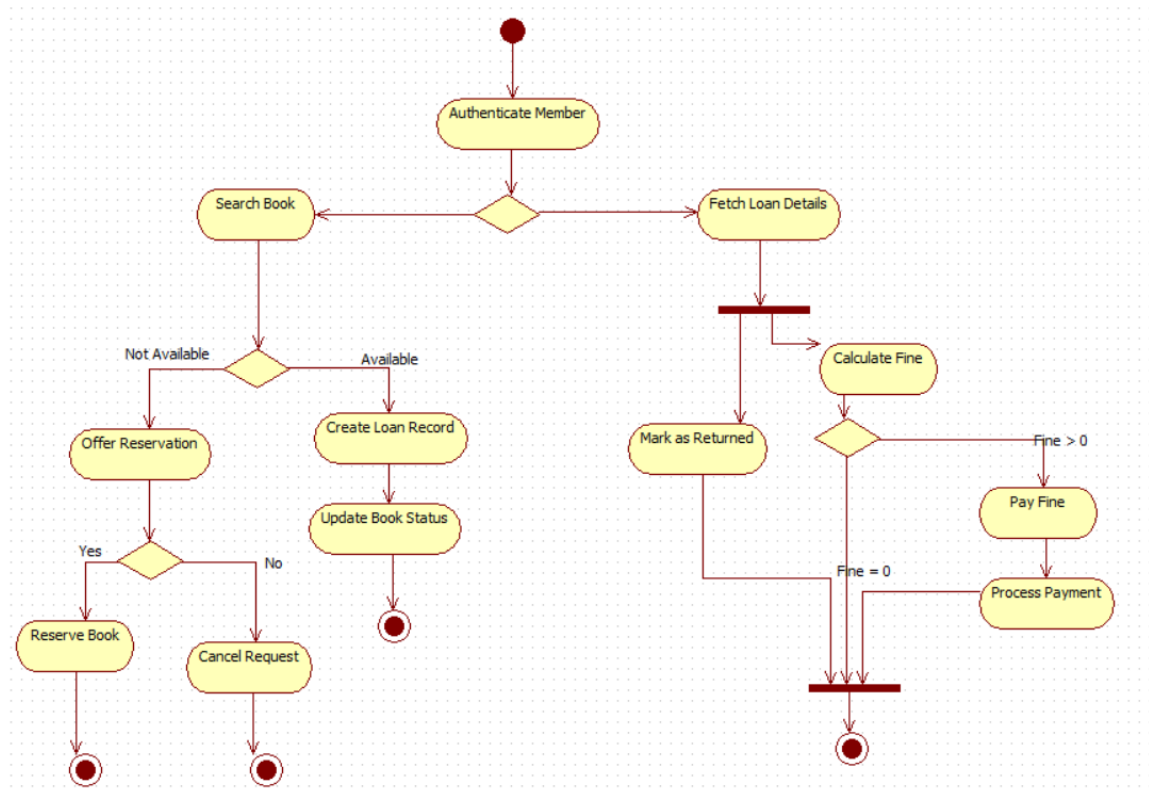


Fig. 3.5 Activity Model

Start & Authentication

- Activity begins at the initial node.
- **Authenticate Member** is executed prior to any further actions.

Search & Availability Check

- Member initiates **Search Book**.
- A decision node evaluates:
 - **Available** → proceed with borrowing.
 - **Not Available** → reservation option.

If Book Is Available

- Execute **Create Loan Record**.
- Perform **Update Book Status** to reflect borrowing.
- Flow terminates at an end node.

If Book Is Not Available

- The system performs **Offer Reservation**.
- A decision node evaluates:
 - **Yes** → execute **Reserve Book** → end.
 - **No** → execute **Cancel Request** → end.

Fetch Loan Information

- From authentication, flow may proceed to **Fetch Loan Details**.
- A synchronization bar splits the return path.

Return Processing

- Execute **Mark as Returned**.
- Parallel path executes **Calculate Fine**.

Fine Decision

- Decision node evaluates:
 - **Fine = 0** → flow joins directly to final synchronization.
 - **Fine > 0** → execute **Pay Fine**, followed by **Process Payment**, then join.

End

- Both return paths converge at a synchronization bar followed by the final node.

4. Stock Maintenance System

4.1 SRS

Problem Statement

A software for Stock Maintenance System.

1. Introduction

1.1 Purpose

To maintain and manage stock records efficiently, saving time and effort.

1.2 Scope

The system helps organizations track available stock, update sale/purchase details, and generate reports.

1.3 Overview

The software provides automated stock management for businesses.

2. General Description

- The system will maintain track of available stock.
- It can generate reports and update stock details.
- It will reduce manual labour and save time.

3. Functional Requirements

- Add new stock item
- Update stock on sale/purchase
- Generate daily/monthly reports
- Notify price drops

4. Interface Requirements

- UI for admin/staff
- Stock search option
- Database for records

5. Performance Requirements

- Must handle at least 2000 transactions/day
- Response time less than 2 seconds
- High fault tolerance

6. Design Constraints

- Should be cross-platform
- Concurrency control

7. Non-Functional Attributes

- Data security and encryption
- Backup and recovery support
- Scalable for large user base

8. Preliminary Schedule & Budget

Phase	Duration (weeks)	Cost (₹)
Requirements Gathering & Analysis	2	15,000
System Design	2	20,000
Development	4	80,000

Testing	2	15,000
Deployment	1	10,000
Maintenance & Documentation	1	5,000
Contingency (10%)	—	14,500
Total Estimated Budget	12 weeks	₹1,59,500 (~₹1.6 Lakhs)

4.2 Class Diagram:

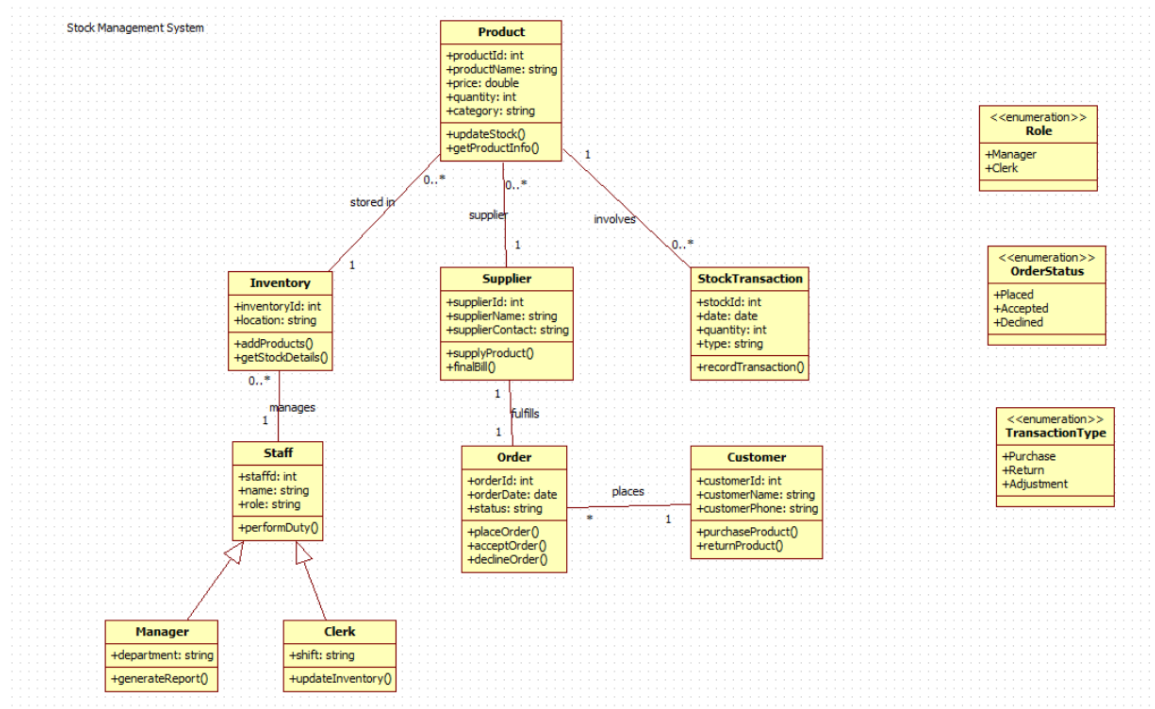


Fig 4.1: Class Diagram

Description

The class diagram represents the structure of the Stock Maintenance System, showing the key entities and their interactions.

Classes

- Product – Contains product details and supports stock updates.
- Inventory – Stores products and maintains stock information.
- Supplier – Provides products to the business and issues bills.
- Order – Represents purchase orders placed by customers.
- Customer – Places orders and purchases or returns products.
- StockTransaction – Records individual stock movement transactions.

- Staff – Handles system operations; specialized into Manager and Clerk.

Enumerations

- Role – Manager, Clerk
- OrderStatus – Placed, Accepted, Declined
- TransactionType – Purchase, Return, Adjustment

Relationships

- Product–Inventory: Products are stored in inventory to maintain organized stock levels.
- Product–Supplier: Products are supplied by suppliers, ensuring continuous stock availability.
- Product–StockTransaction: Stock transactions record movements of products to track purchase/return activity.
- Inventory–Staff: Inventory is managed by staff to ensure proper stock handling and updates.
- Supplier–Order: Suppliers fulfill orders placed by the system to replenish inventory.
- Customer–Order: Customers place orders to purchase products from the system.
- Order–StockTransaction: Each order involves stock transactions to update quantities accordingly.
- Staff–Manager/Clerk: Staff roles are specialized into manager and clerk for reporting and inventory update responsibilities.

4.3 State Diagram:

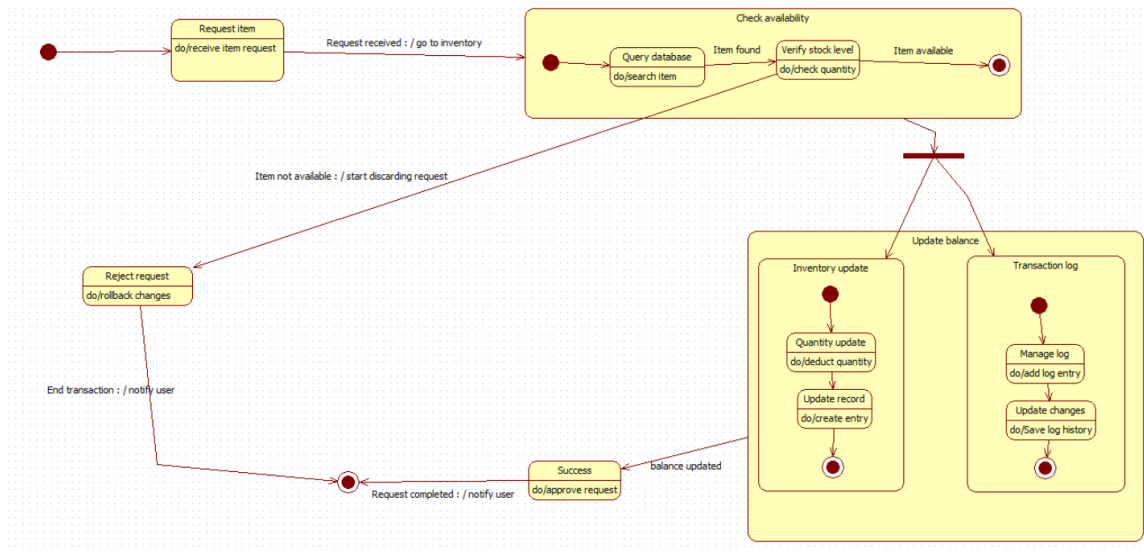


Fig 4.2: State Diagram

Description

The state diagram illustrates the flow of activities in the **Stock Request Process**, showing how the system processes an item request from initiation to completion.

States

- **Request Item:** The user submits an item request to the system.
- **Check Availability (Composite State):** The system searches inventory and verifies stock quantity.
 - Includes substates: **Query Database** and **Verify Stock Level**.
- **Reject Request:** Triggered when the item is not available; the system rolls back any partial changes.
- **Update Balance (Composite State):** Executed when the item is available, updating both stock records and transaction logs.
 - Synchronous Substates: **Inventory Update** and **Transaction Log**.
- **Success:** The request is approved and finalized.

- **End Transaction:** The system notifies the user of completion.

Flow

8. User submits a request → system checks availability.
9. If the item is unavailable → request is rejected.
10. If available → inventory is updated and transaction logs recorded.
11. The process ends by notifying the user of success or rejection.

4.4 Use Case Diagram:

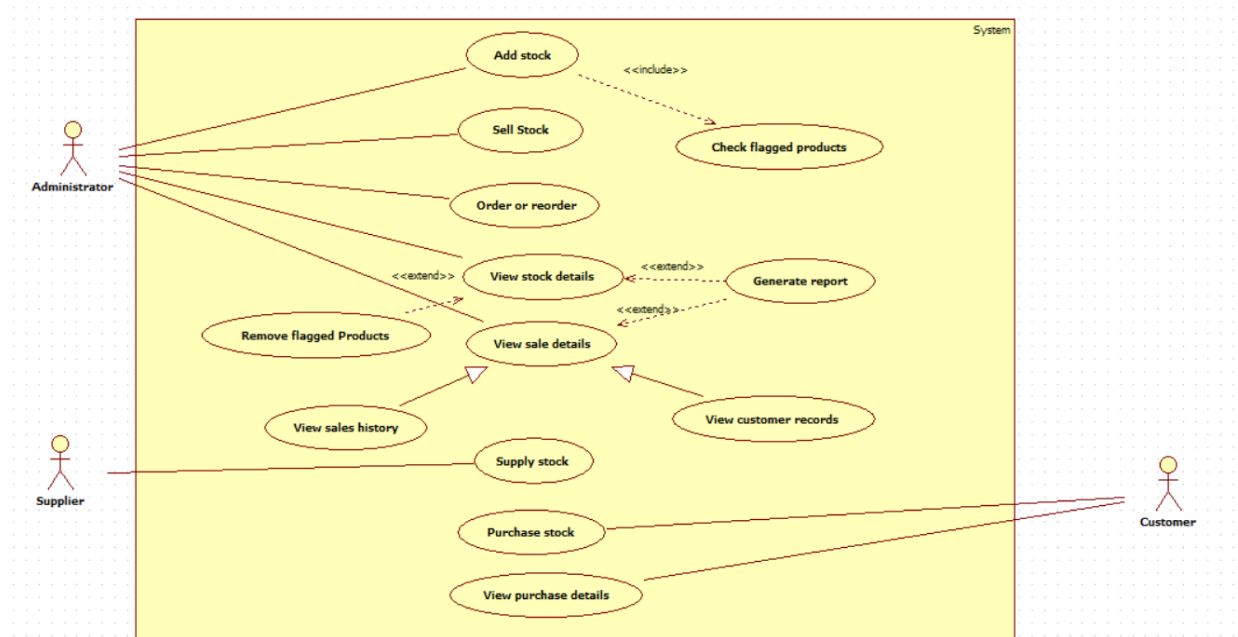


Fig 4.3: Use Case Diagram

Description

The use case diagram illustrates the interactions between the key actors of the Stock Maintenance System and the various system functionalities.

Actors

- Administrator – Manages stock operations, monitors flagged products, and generates reports.
- Supplier – Supplies stock to the system and updates purchase-related information.
- Customer – Purchases stock and views purchase details.

Use Cases

- Add Stock, Sell Stock, Order or Reorder – Core inventory operations performed by the Administrator.
- Check Flagged Products – Included when adding stock to ensure quality control.

- View Stock Details, View Sale Details, View Customer Records – Administrative monitoring functions.
- Remove Flagged Products – Extends stock viewing for removing defective items.
- Generate Report – Extends viewing functions to create summaries and analytics.
- Supply Stock – Supplier provides stock to the system.
- Purchase Stock – Customer initiates stock purchase.
- View Sales History, View Purchase Details – For reviewing transaction records.

4.5 Sequence Diagram:

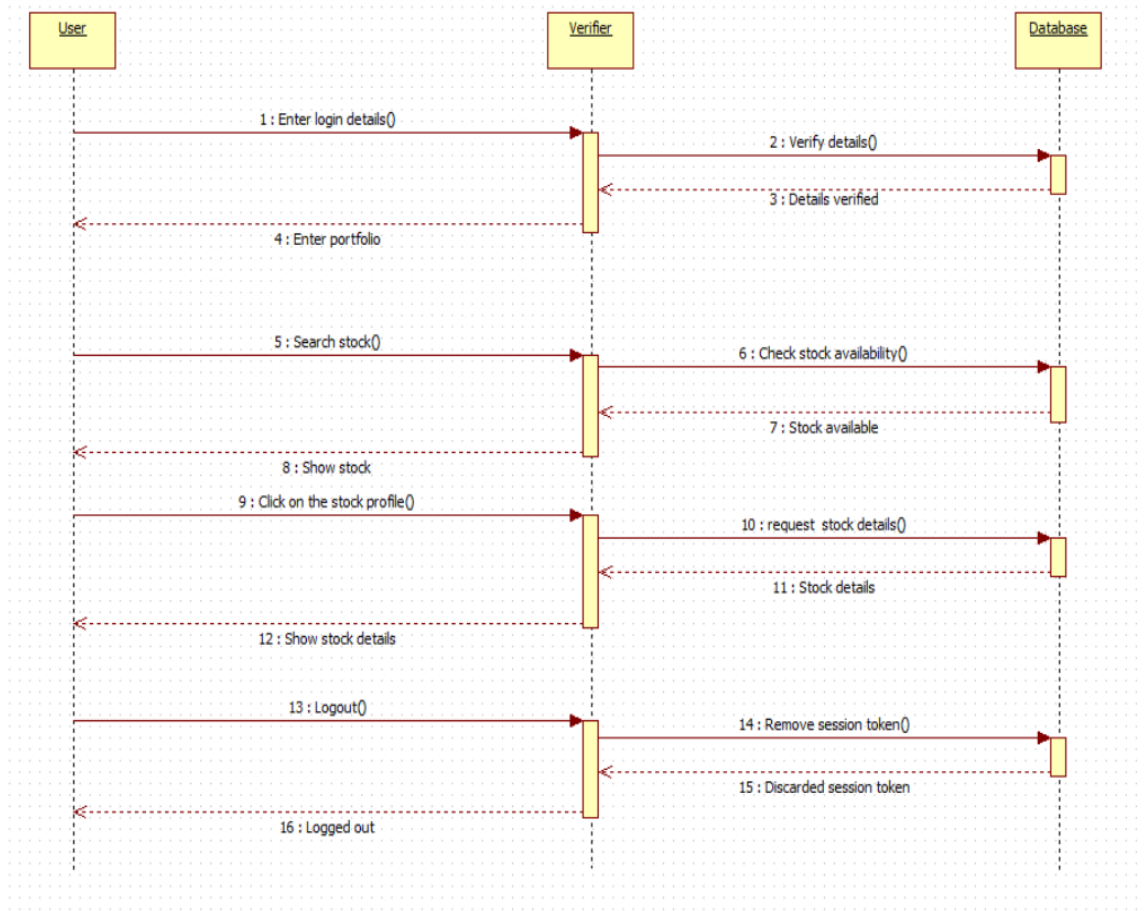


Fig 4.4: Sequence diagram

Description:

The sequence diagram illustrates the flow of interactions in the stock viewing process, from user login to logout, showing how the Verifier and Database coordinate to retrieve and display stock information.

Objects:

- User: Initiates the login, searches for stock, and requests details.
- Verifier: Validates login details, processes stock requests, and coordinates with the database.
- Database: Stores user credentials and stock data; returns verification and stock information.

Flow:

1. The User enters login details, which are forwarded to the Verifier.
2. The Verifier checks the credentials with the Database, which verifies and returns the authentication result.
3. After login, the User searches for stock, and the Verifier sends a stock availability request to the Database.
4. The Database confirms availability, and the User selects a stock profile for more details.
5. The Verifier requests detailed stock data from the Database and returns it to the User.
6. The User logs out, and the Verifier removes the session token from the Database.
7. The Database confirms session removal, completing the logout process.

4.6 Activity Diagram:

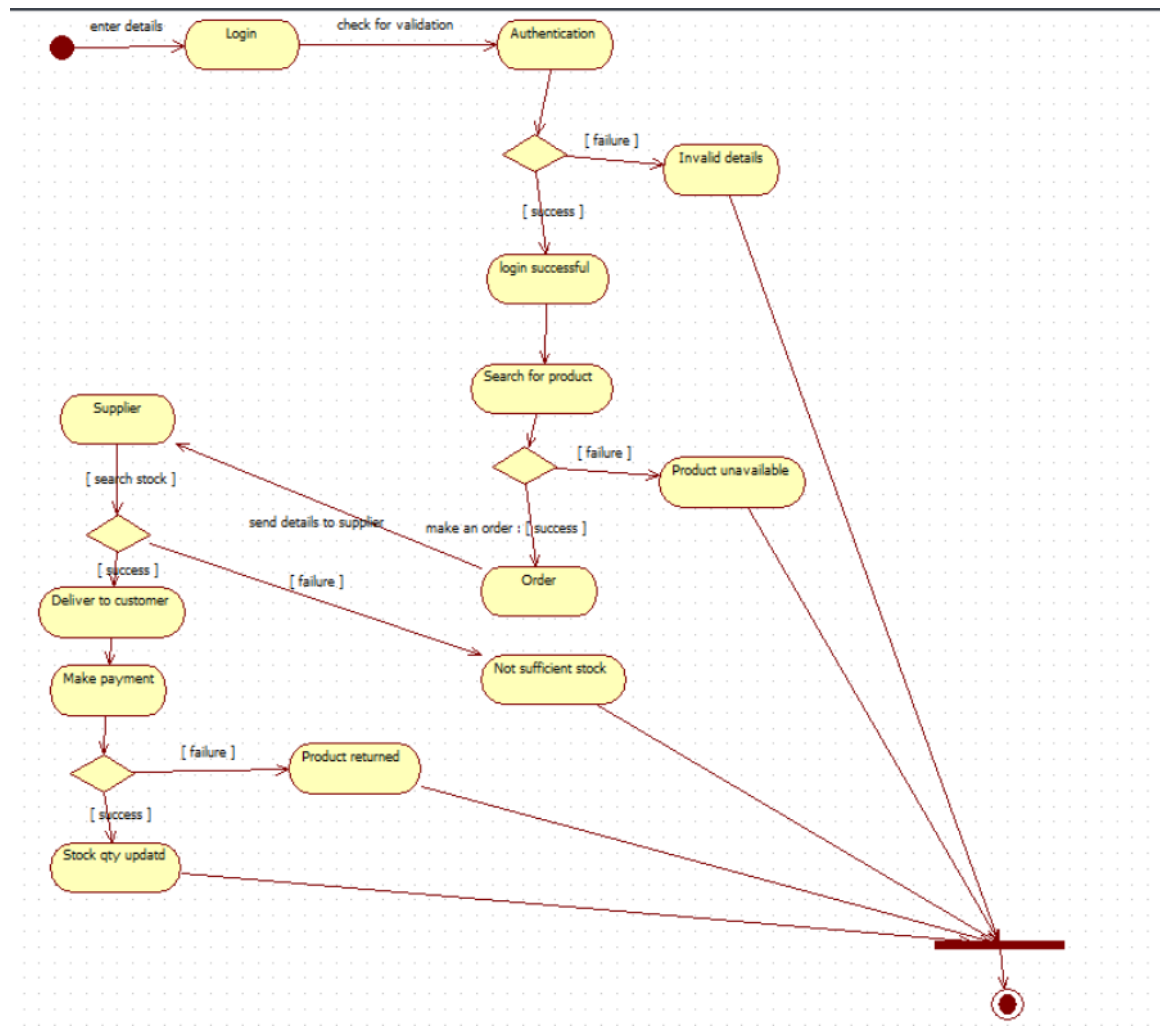


Fig 4.5: Activity Diagram

Description:

The activity diagram illustrates the flow of operations in the stock ordering and purchasing process, starting from user login and continuing through product search, ordering, supplier interaction, and payment or return handling.

Activities:

- Login: User enters login details to access the system.
- Authentication: Validates user credentials.
- Search for Product: User searches for required items.

- Supplier Interaction: Supplier checks stock availability and returns results.
- Order: User places an order for the product.
- Deliver to Customer: Product is delivered if sufficient stock is available.
- Make Payment: Customer completes payment for the delivered product.
- Product Returned: Triggered when payment fails or order is reversed.
- Stock Quantity Update: Inventory is updated after a successful return or delivery.

Flow:

1. The user enters details and proceeds to Login, followed by Authentication.
2. If authentication fails, the user is redirected to Invalid Details; if successful, they proceed to Search for Product.
3. The system checks availability; if unavailable, it moves to Product Unavailable.
4. If available, the user places an Order, which is forwarded to the Supplier.
5. Supplier checks stock; on success, the product is delivered to the customer.
6. Customer proceeds to Make Payment; on success, the process ends.
7. If payment fails or stock is insufficient, the process moves to Product Returned, followed by Stock Quantity Updated, and then ends.

5. Passport Automation System

5.1 SRS

Problem Statement

A software for Passport Automation System.

1. Introduction

1.1 Purpose

To develop an automated system that streamlines passport application processing, minimizes manual errors, and accelerates issuance while ensuring data security and regulatory compliance.

1.2 Scope

The system allows online passport application submission, document verification, appointment scheduling, biometric data capture, application tracking, and passport issuance. It provides access for applicants, verification officers, and administrators.

1.3 Overview

The software automates the complete passport lifecycle—from application intake to final delivery—enhancing efficiency, transparency, and security.

2. General Description

- Supports online application submission with document upload.
- Biometric data collection (fingerprints, photo).
- Automated appointment booking and notification system.
- Enables verification, approval/rejection workflows.
- Allows real-time status tracking and alerts.
- Generates administrative reports.

3. Functional Requirements

- User registration and secure login
- Application form filling with validation and document upload
- Scheduling and management of biometric/verification appointments
- Verification and approval/rejection by officers
- Passport issuance and dispatch tracking
- Email/SMS notification system
- Administrative reporting

4. Interface Requirements

- Responsive web interface (desktop & mobile)
- Secure document upload support
- Integration with biometric devices (cameras, fingerprint scanners)
- APIs for government databases & postal services
- HTTPS-secured data transfer

5. Performance Requirements

- Support minimum 5,000 concurrent users
- Average application processing time ≤ 48 hours
- Real-time status updates
- System uptime 99.9% (excluding maintenance)

6. Design Constraints

- Must follow government security and privacy regulations
- Encrypted storage for sensitive data

- Compatible with national ID database integration
- Modular structure to allow future upgrades
- Hardware compatibility with biometric devices

7. Non-Functional Attributes

- High security & confidentiality of user data
- User-friendly UI with multilingual support
- Scalable to support future user growth
- Reliable backup & disaster recovery
- Maintainable with proper documentation

8. Preliminary Schedule & Budget

Phase	Duration (weeks)	Cost (₹)
Requirements Gathering	3	3,50,000
System Design	4	4,50,000
Development	12	20,00,000
Testing	4	5,00,000
Deployment	2	2,00,000
Training & Documentation	1	1,00,000
Contingency (10–15%)	-	3,50,000
Total	26 weeks	39,50,000

5.2 Class Diagram:

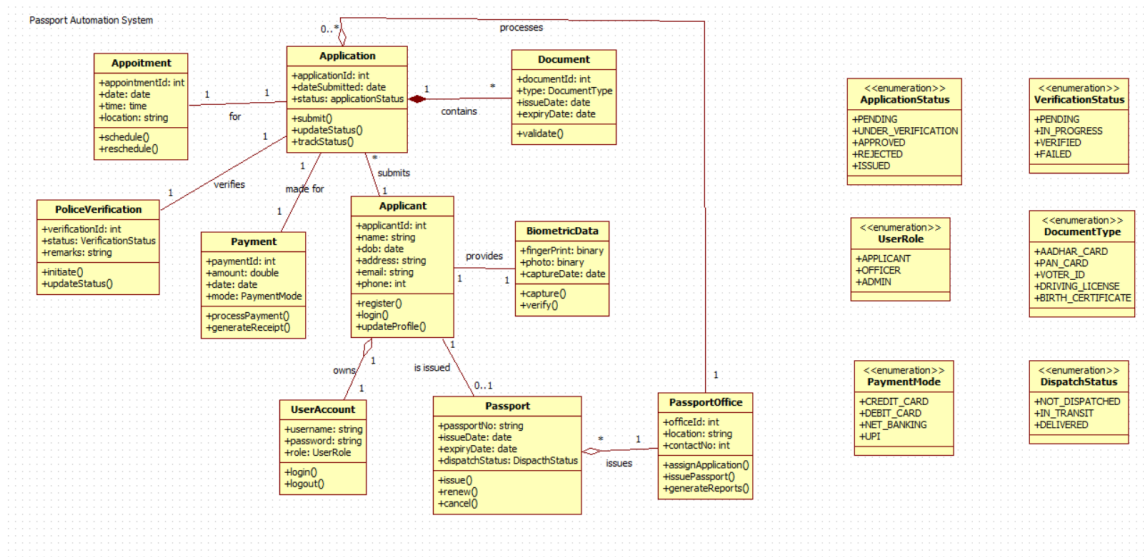


Fig 5.1 Class Diagram

Description

The class diagram represents the structure of the Passport Automation System, showcasing the primary entities involved in the passport application, verification, and issuance process, along with their interactions.

Classes

- **Applicant** : Stores applicant details such as name, date of birth, address, email, and phone number. Supports operations like registration, login, and updating profile.
- **UserAccount** : Manages authentication using username, password, and user role. Provides login and logout functionalities.
- **Application** : Represents the passport application submitted by the applicant. Tracks submission date and current status. Includes methods to submit, update, and track application progress.
- **Appointment** : Used to schedule biometric and verification appointments. Contains date, time, and location, with options to schedule or reschedule.
- **Document** : Contains the supporting documents submitted by the applicant. Stores document type, issue date, and expiry date, and includes validation functionality.
- **BiometricData** : Holds biometric details such as fingerprint and photo, along with the capture date. Supports biometric capture and verification.
- **PoliceVerification** : Manages police verification status and remarks. Includes methods to initiate and update verification progress.

- **Payment** : Represents payment details including amount, date, and mode of payment. Includes operations for processing payment and generating payment receipt.
- **Passport** : Stores passport information such as passport number, issue date, expiry date, and dispatch status. Supports issuing, renewing, and cancelling the passport.
- **PassportOffice** : Handles administrative functionalities including application assignment, passport issuance, and report generation.

Enumerations

- **ApplicationStatus** – Pending, Under_Verification, Approved, Rejected, Issued
- **VerificationStatus** – Pending, In_Progress, Verified, Failed
- **UserRole** – Applicant, Officer, Admin
- **DocumentType** – Aadhar_Card, PAN_Card, Voter_ID, Driving_License, Birth_Certificate
- **PaymentMode** – Credit_Card, Debit_Card, Net_Banking, UPI
- **DispatchStatus** – Not_Dispatched, In_Transit, Delivered

Relationships

- **Applicant – UserAccount**: An applicant owns one user account.
- **Applicant – Application**: One applicant submits one or more applications.
- **Application – Appointment**: Each application is associated with one appointment.
- **Application – Document**: An application contains one or more documents.
- **Application – Payment**: Payments are made for applications.
- **Application – PoliceVerification**: Each application undergoes police verification.
- **Applicant – BiometricData**: Biometric data is provided once per applicant.
- **Passport – Applicant**: A passport is issued to a verified applicant
- **PassportOffice – Application**: The passport office processes applications.
- **PassportOffice – Passport**: The passport office issues one or more passports.

5.3 State Diagram:

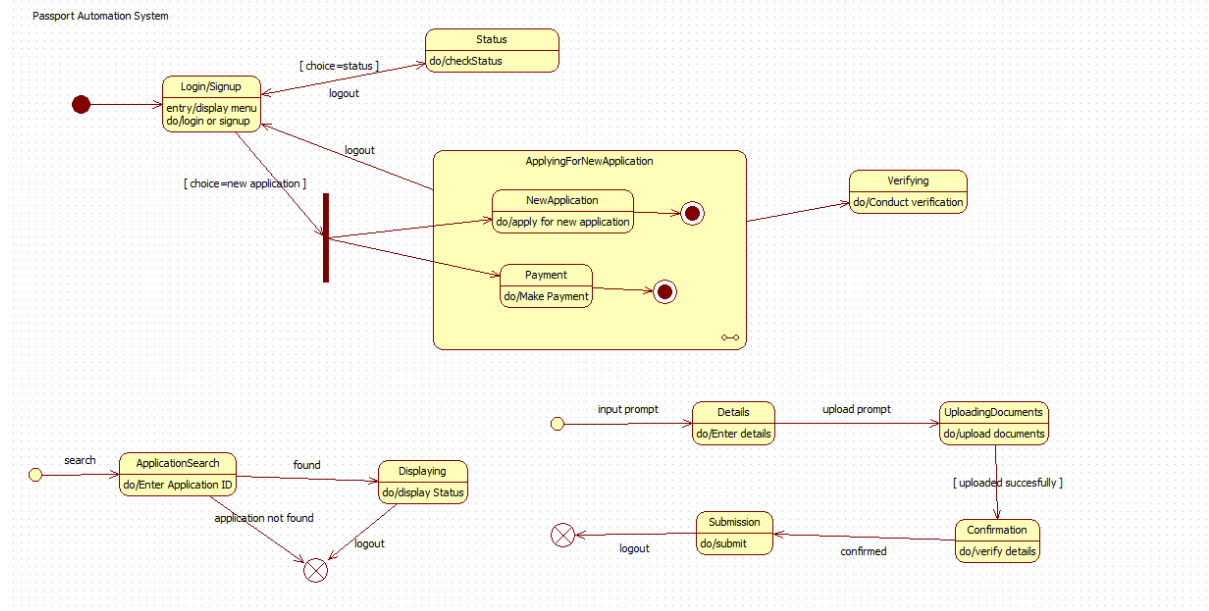


Fig 5.2 State Diagram

Description

The state diagram illustrates the sequential flow of the Passport Automation System, highlighting how an applicant interacts with the system throughout the passport application lifecycle. It demonstrates system behavior from login to document uploading, verification, and final submission. The diagram depicts decision points, composite states, and termination scenarios such as logout and application not found.

States

- **Login/Signup:** The process starts when the user logs in or signs up. The menu is displayed, allowing the user to either apply for a new passport or check the application status. Logout transitions directly to termination.
- **Status:** Triggered when the user chooses to check the application status. The system fetches and displays the current status of the application.
- **Applying For New Application (Composite State):** This composite state represents the complete passport application process and includes two substates:
 - **NewApplication** – The applicant fills out initial application details.
 - **Payment** – The applicant completes payment for processing the application.
- **Details:** The user enters personal and application-related information when prompted.
- **UploadingDocuments** : The applicant uploads compulsory supporting documents. On successful upload, the system transitions to confirmation.

- **Confirmation:** The uploaded documents and details are verified. If confirmed, the process advances to submission.
- **Submission:** Final submission occurs after all details and documents are validated. Unsuccessful or incorrect input may return to previous states or lead to logout.
- **Verifying:** This state handles the document and biometrics verification process post submission.
- **ApplicationSearch:** If the process starts with "search," the user enters the application ID.
 - If the ID is found → moves to "Displaying".
 - If not found → leads to termination.
- **Displaying:** Shows the current status of the application.
- **End State:** Occurs upon user logout or if the application search fails.

Flow

- The user logs in or signs up; unsuccessful attempts or logout lead to termination.
- The applicant selects:
 - Check status → moves to Status.
 - New application → transitions into ApplyingForNewApplication.
- In the composite state, the applicant fills in the application form (NewApplication) and completes payment (Payment).
- The system then prompts the user to enter personal details → upload documents.
- After a successful upload, details are verified in Confirmation and submitted via Submission.
- Once submitted, the system moves to *Verifying* for further processing.
- Alternatively, the user may search for an existing application using *ApplicationSearch*. If found, the system displays status; otherwise, the process ends.
- Any logout at multiple checkpoints ends the session.

5.4 Use Case Diagram:

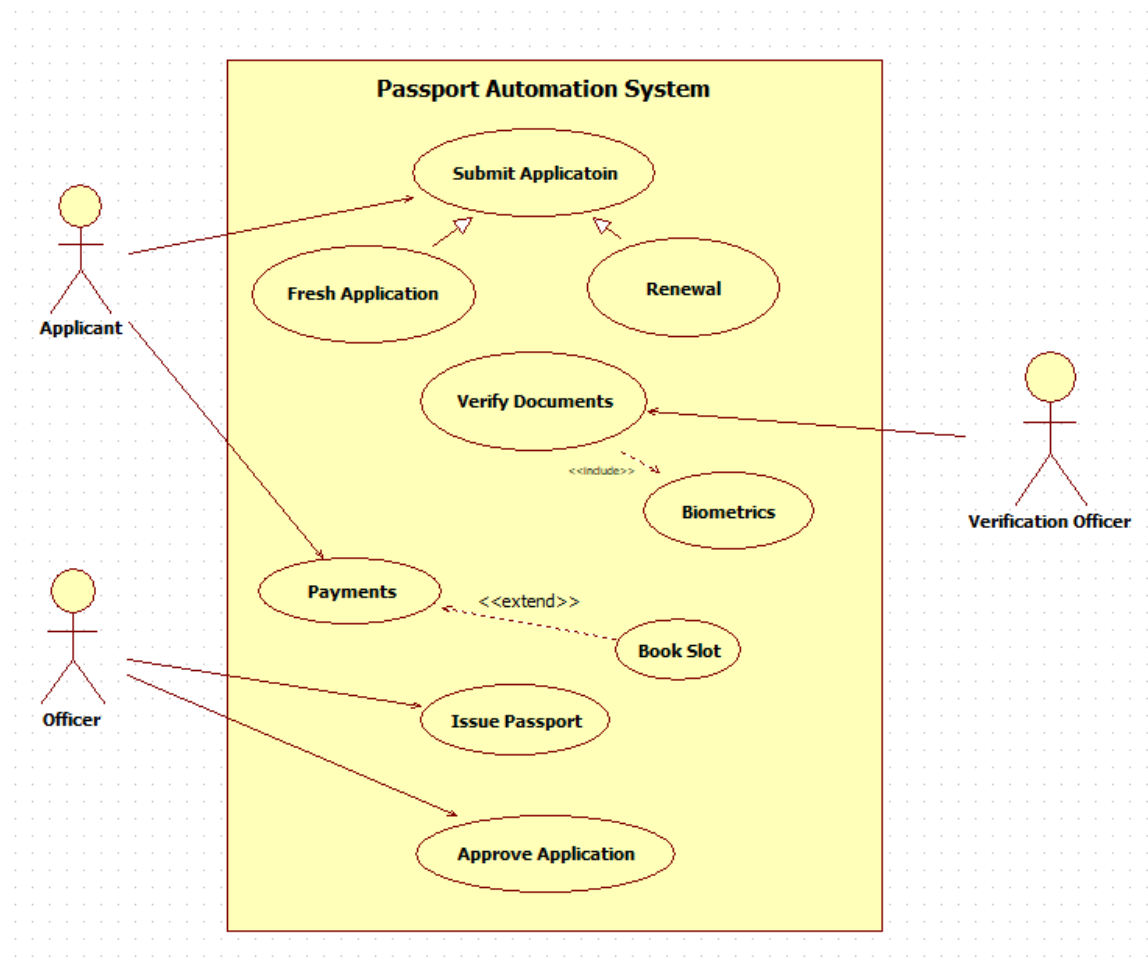


Fig 5.3 Use Case Diagram

Description

The use case diagram represents the interactions between the main actors involved in the Passport Automation System and the functional services provided by the system. It demonstrates how applicants, officers, and verification officers interact with the system to apply, process, and issue passports efficiently.

Actors

- **Applicant** – Initiates passport applications, uploads required documents, completes payment, and books biometric appointments.

- **Officer** – Reviews applications, approves or rejects them, and issues passports upon successful verification.
- **Verification Officer** – Validates the applicant's documents and performs biometric verification.

Use Cases

Applicant

- **Submit Application**
 - Fresh Application – For new passport registrations.
 - Renewal – For renewing existing passports.
- **Verify Documents**
 - Includes → Biometrics – Capturing fingerprint and photograph.
- **Payments**
 - Extends → Book Slot – Scheduling a slot for biometric verification.

Officer

- **Approve Application** – Validates and approves/rejects passport applications.
- **Issue Passport** – Generates and dispatches the passport after approval.

Verification Officer

- **Biometrics** – Performs biometric verification during the booked slot.
- **Verify Documents** – Checks the authenticity and validity of submitted documents.

5.5 Sequence Diagram:

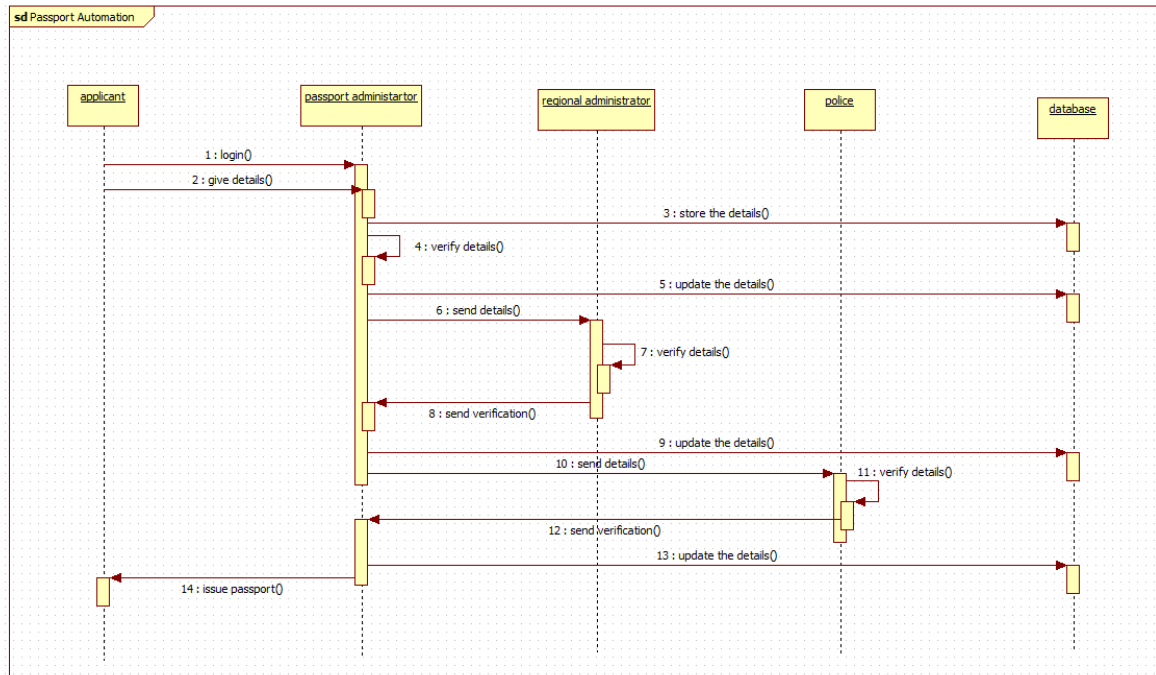


Fig 5.4 Sequence Diagram

Description

The sequence diagram illustrates the interaction flow during the passport application process, from applicant login to passport issuance. It shows how the Passport Administrator, Regional Administrator, Police, and Database collaborate to verify applicant details and complete the application process.

Objects

- **Applicant:** Initiates the process by logging in and submitting application details.
- **Passport Administrator:** Receives and verifies application details, coordinates further processing.
- **Regional Administrator:** Performs regional-level verification and updates the details.
- **Police:** Conducts physical verification of the applicant.
- **Database:** Stores and updates all application, verification, and approval records.

Flow

1. The Applicant logs in and submits application details to the Passport Administrator.
2. The Passport Administrator stores the details in the Database.
3. The administrator verifies the submitted information.
4. Verified details are updated in the Database.
5. The Passport Administrator sends the details to the Regional Administrator for further verification.
6. The Regional Administrator verifies the details and updates the records in the Database.
7. The Regional Administrator sends verification results back to the Passport Administrator.
8. The Passport Administrator sends the verification request to the Police.
9. The Police verify applicant details and update the status in the Database.
10. The Police return verification confirmation to the Passport Administrator.
11. The Passport Administrator updates the final details in the Database.
12. After all verifications are successfully completed, the Passport Administrator proceeds to issue the passport.
13. The Applicant receives the passport, completing the process.

5.6 Activity Diagram

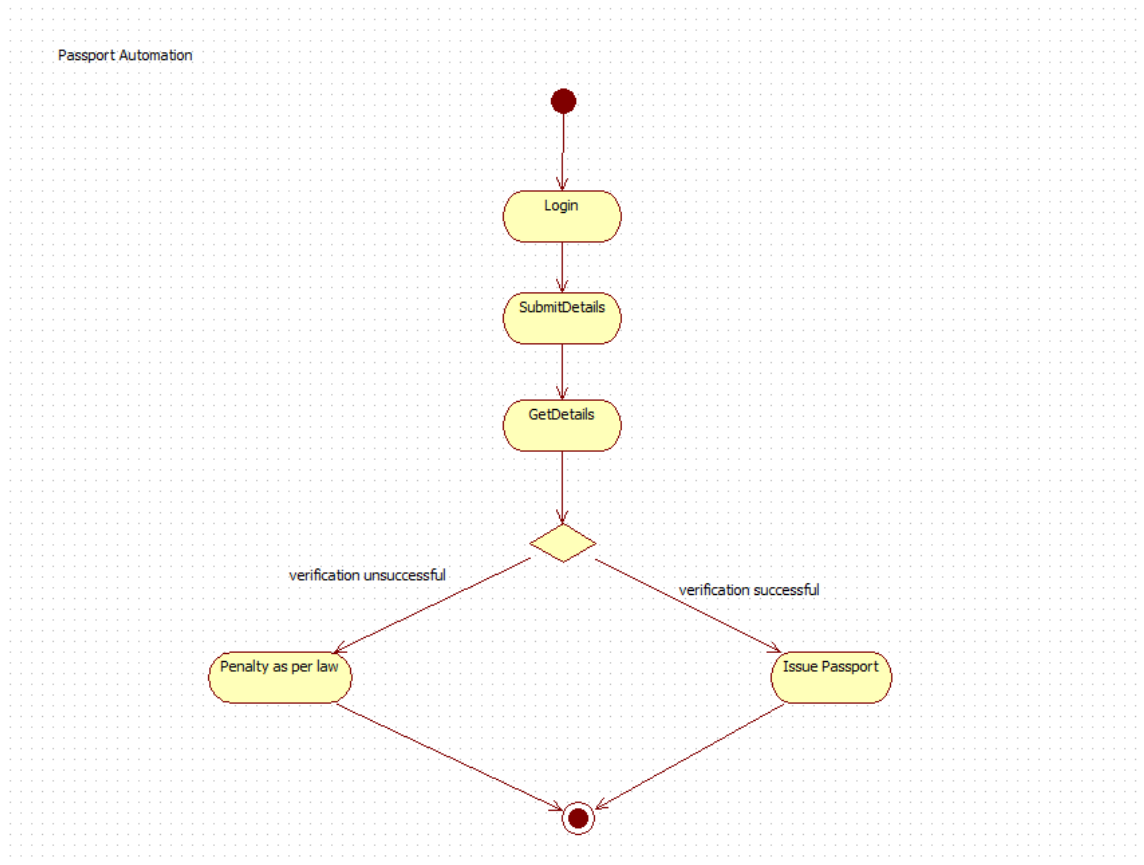


Fig 5.5 Activity Diagram

Description

The activity diagram illustrates the workflow of the passport application process, starting from user login and continuing through application submission, detail retrieval, verification, and final decision, either passport issuance or legal action.

Activities

- **Login:** The applicant accesses the system by entering valid login credentials.
- **Submit Details:** The user provides application and personal information required for passport processing.
- **Get Details:** System retrieves and prepares the provided details for validation.
- **Verification Decision:** A decision node checks whether verification (document and biometric validation) is successful.

- **Issue Passport:** If verification is successful, the passport is issued to the applicant.
- **Penalty as per Law:** If verification fails (e.g., incorrect or fraudulent submission), penalties are imposed based on legal regulations.

Flow

1. The process begins with the Login activity.
2. After successful login, the user proceeds to Submit Details, entering the required application data.
3. The system performs Get Details, collecting and preparing the submitted information.
4. At the Decision node, verification results are evaluated:
 - If verification is successful, the process moves to Issue Passport, and the application is approved.
 - If verification is unsuccessful, the process transitions to Penalty as per Law, applying legal consequences.
5. The process concludes after either passport issuance or penalty application.